**ACADEMIC & TECHNICAL ARCHITECTURE REPORT**

# Automated Chest Radiograph Diagnostic System for Tuberculosis & Pneumonia

A Deep Learning, Real-Time OPD Token Queue & Tele-Radiology Web Ecosystem

| Attribute | Detail |
|---|---|
| Project Title | TB & Pneumonia AI Medical Screening and Clinic Management System |
| System Domain | Medical Imaging, Computer Vision, Telehealth & OPD Workflow Automation |
| AI Backbone | DenseNet-121 (3-class: Normal / Tuberculosis / Pneumonia) + Grad-CAM explainability |
| Computer Vision | OpenCV 6-stage pre-inference sanity gate (geometry, focus, dynamic range, chrominance, skin & biometric cascade, structural edge verification) |
| Backend Core | FastAPI 0.115.6 (Python 3.9+), Uvicorn 0.34.0 ASGI, WebSockets, SQLAlchemy 2.0.36 ORM |
| Frontend Core | React 18 + Vite 5, Tailwind CSS 3, Framer Motion 10, Axios 1.6 interceptors |
| Database Stack | PostgreSQL (production) / SQLite (isolated test harness) |
| Security Model | OAuth2 password flow, HS256 JWT bearer tokens, bcrypt salting, doctor-only self-registration via single-use access codes |
| Automated Tests | 61 passing backend tests across 5 modules (auth security, media access, queue, admin integrity, contact settings) |
| Codebase Size | 30 frontend modules (17,083 lines) + 14 backend modules (3,689 lines) |
| API Surface | 55 route handlers (54 REST + 1 WebSocket) across two routers, all under the /api prefix |
| Academic Origin | BS Software Engineering Final Year Project, University of Swat |

**Executive Overview.** This document is the exhaustive technical specification of the project repository. It documents every architectural layer: how the React single-page application interfaces with the FastAPI backend, the authentication token lifecycle, the DenseNet-121 model and its explainability layer, the six-stage computer vision pre-screening gate, the relational schema, real-time WebSocket queue synchronisation, authenticated medical media serving, a page-by-page interface analysis, a complete endpoint directory, deployment procedures, the security model, and the automated test suite that guards all of it.

**Revision note.** This revision supersedes the initial issue. It corrects the technology version matrix against the pinned dependency manifests, replaces the previously documented two-stage image validator with the six-stage gate actually present in the codebase, removes an inaccurate description of unauthenticated static media mounting, corrects the registration workflow to reflect the doctor-only public sign-up policy, expands the endpoint directory to all 55 route handlers, and adds Sections 12 through 15 covering security, testing, the engineering change log, and known limitations. A subsequent hardening pass removed a duplicated route handler, enforced doctor-only access on the clinical dashboard, removed a plaintext database credential from source, closed a patient-identity leak on the public waiting-room queue, and granted administrators read and delete oversight of predictions and appointments. All endpoint and file line counts in this revision were recomputed from the working tree.

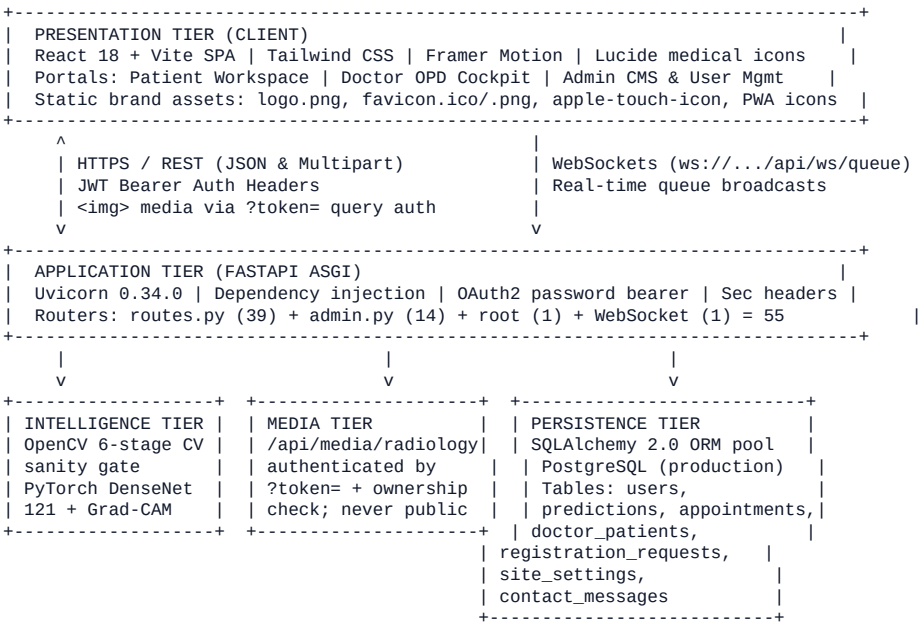
# Table of Contents


| # | Section | Page |
|---|---|---|
| 1 | High-Level System Architecture & Design Philosophy | 3 |
| 2 | Complete Technology Stack Matrix (pinned versions) | 4 |
| 3 | How Frontend and Backend Connect (Axios Interceptors, JWT, WebSockets, Multipart, CORS) | 6 |
| 4 | Deep Learning Model Pipeline & the Six-Stage Computer Vision Sanity Gate | 8 |
| 5 | Relational Database Design, Schema Reference & Cascading Integrity | 10 |
| 6 | File-by-File Repository Breakdown & Architectural Map | 11 |
| 7 | Comprehensive Frontend Page-by-Page Breakdown | 13 |
| 8 | Role-Based Portals & Functional Workflows | 14 |
| 9 | Complete RESTful API & WebSocket Endpoint Directory (55 route handlers) | 16 |
| 10 | Step-by-Step Transaction Lifecycles | 20 |
| 11 | Deployment, Environment Configuration & Troubleshooting | 21 |
| 12 | Security Architecture & Access Control Model | 23 |
| 13 | Automated Test Suite & Verification Evidence | 24 |
| 14 | Appendix A: Engineering Change Log | 25 |
| 15 | Appendix B: Known Limitations & Recommended Next Steps | 27 |
| 16 | Appendix C: Viva Voce Question Bank | 28 |


Page references are resolved automatically in a second layout pass, so the contents table above is authoritative for this generated revision.

# 1. High-Level System Architecture & Design Philosophy

The system follows a decoupled, multi-tier architecture. The presentation tier (React SPA) is completely isolated from the application tier (FastAPI), communicating strictly through standardised RESTful JSON payloads, multipart file streams, and bidirectional WebSocket events. No server-side session store exists; every request is independently authenticated.




## Key Architectural Tenets

- **Stateless web core.** The API holds no in-memory user sessions. Authentication is encapsulated in signed JWTs verified on every request. The single deliberate exception is the doctor registration-code burn set, which is a per-process replay guard backed by a uniqueness constraint in the database (see Section 12).
- **Fail-fast medical sanity gate.** Low-quality, corrupted, non-radiological or human-subject photographs are rejected before any neural inference, across six independent OpenCV stages (Section 4.5).
- **No fabricated diagnoses.** If the DenseNet weights are missing or fail to load, the service refuses to predict. The `/api/predict` route returns HTTP 503 rather than emitting a random label, and the frontend disables its analysis control until `/api/model-status` reports a connected model.
- **Real-time event broadcast.** Waiting-room displays, doctor call registers and patient handsets synchronise over WebSockets instead of database polling.
- **Role-based access control.** Explicit role checks exist on both the frontend router guards and the backend route dependencies.
- **Private-by-default medical media.** Diagnostic radiographs are never served as public static files. They are streamed only through an authenticated route that re-checks record ownership per request (Section 3.6).

# 2. Complete Technology Stack Matrix

Every version below is transcribed directly from the pinned manifests (`backend/requirements.txt` and `frontend/package.json`), not from memory. Caret ranges are the declared compatible ranges; the resolved lockfile version is given where it differs.

## *2.1 Backend & Intelligence*

| Layer | Technology | Pinned Version | Role & Clinical Justification |
|---|---|---|---|
| Backend web | FastAPI | 0.115.6 | Async ASGI framework with dependency injection and auto-generated OpenAPI docs. |
| ASGI server | Uvicorn (standard) | 0.34.0 | Production ASGI server with uvloop and httptools acceleration. |
| ORM | SQLAlchemy | 2.0.36 | Declarative relational mapping and connection pooling. |
| DB driver | psycopg2-binary | 2.9.10 | PostgreSQL wire-protocol driver. |
| Validation | Pydantic | 2.10.4 | Runtime payload validation and serialisation. |
| Settings | pydantic-settings | 2.7.0 | Typed environment loading with fail-fast production guards (Section 12.1). |
| Migrations | Alembic | 1.14.0 | Declared in requirements but not initialised: the repository contains no alembic.ini or migrations directory, and the schema is created by Base.metadata.create_all() at startup. See Section 15. |
| Uploads | python-multipart | 0.0.20 | Multipart form parsing for radiograph streaming. |
| Deep learning | PyTorch | 2.5.1 | DenseNet-121 forward pass and Grad-CAM backprop. |
| Vision models | Torchvision | 0.20.1 | DenseNet-121 definition and 224x224 preprocessing transforms. |
| Computer vision | opencv-python-headless | 4.10.0.84 | Six-stage radiograph validation and heatmap compositing. |
| Imaging | Pillow | 11.0.0 | Image decoding and re-encoding. |
| Numerics | NumPy | 2.1.3 | Tensor marshalling and activation maths. |
| Password hash | bcrypt | 4.2.1 | Salted Blowfish key derivation. |
| JWT | python-jose | 3.3.0 | HS256 token signing and verification. |
| Realtime | websockets | 14.1 | ASGI WebSocket transport. |
| Config | python-dotenv | 1.0.1 | Local environment file loading. |
| HTTP client | httpx | 0.28.1 | TestClient transport for the automated suite. |
| Testing | pytest | 8.3.4 | Backend test runner. |

## *2.2 Frontend*

| Layer | Technology | Declared Range | Role & Clinical Justification |
|---|---|---|---|
| UI runtime | React / React DOM | ^18.2.0 | Hooks-driven concurrent rendering. |
| Build | Vite | ^5.0.0 | ESM dev server with sub-second HMR; Rollup production bundles. |
| Routing | React Router DOM | ^6.19.0 | Client routing plus ProtectedRoute RBAC guards. |
| Styling | Tailwind CSS | ^3.3.5 | Glassmorphism clinical theme, light and dark modes. |
| Animation | Framer Motion | ^10.16.5 | Card transitions, progress meters, modal choreography. |
| Icons | Lucide React | ^0.292.0 | Healthcare iconography (stethoscope, lungs, activity). |
| HTTP | Axios | ^1.6.2 | Request/response interceptors for automatic JWT injection. |
| Charts | Recharts | ^3.8.1 | Dashboard area and pie charts for disease distribution. |
| Dialogs | SweetAlert2 | ^11.26.24 | Clinical alert and destructive-confirmation modals. |
| Toasts | React-Toastify | ^11.1.0 | Transient non-blocking status notifications. |
| PDF export | html2pdf.js | ^0.14.0 | Client-side A4 medical report compilation. |
| Linting | ESLint | ^8.53.0 | Runs with --max-warnings 0; the repository currently reports zero warnings. |

### *2.3 Infrastructure*

| Component | Technology | Role |
|---|---|---|
| Primary database | PostgreSQL 13 / 18 | ACID relational store with foreign keys and indexes. |
| Test database | SQLite (in-memory) | Isolated fixture database; StaticPool keeps one shared connection. |
| Containers | Docker Compose (v2) | Four-file stack under docker/: backend and frontend Dockerfiles, an Nginx reverse proxy, and the Compose orchestrator. |
| Reverse proxy | Nginx | TLS termination and static SPA fallback for production builds. |

Correction against the previous issue: this matrix previously listed Framer Motion as v11.x when the manifest declares **^10.16.5**, and omitted Recharts, SweetAlert2, html2pdf.js, React-Toastify, psycopg2-binary, pydantic-settings, Alembic, pytest, Pillow, NumPy, websockets, python-dotenv, httpx, python-multipart and the headless OpenCV build.

# 3. How Frontend and Backend Connect

The React to FastAPI integration is engineered for low latency, predictable failure modes and airtight session security. Six mechanisms govern it.

## 3.1 Dynamic API Endpoint Resolution

In `frontend/src/services/api.js` the client derives two base URLs from a single environment variable. `VITE_API_URL` supplies the server origin; any trailing `/api` suffix is stripped so that the media origin is correct, then the API base is re-appended. This yields both `API_BASE_URL` for XHR calls and `MEDIA_BASE_URL` for radiograph files, avoiding duplicated origin configuration.

```
const configuredApiUrl = (import.meta.env.VITE_API_URL || "http://localhost:8000")
  .trim().replace(/\/+$/, '');
const serverBaseUrl = configuredApiUrl.replace(/\/api$/, '');
export const API_BASE_URL     = `${serverBaseUrl}/api`;
export const MEDIA_BASE_URL   = serverBaseUrl;
```

## 3.2 Axios Interceptors & the JWT Protocol

The login and request transaction proceeds as follows:

1. The client posts an identifier (phone or CNIC) and password to `/api/auth/login`.
2. The server verifies the bcrypt hash and issues an HS256-signed JWT carrying `{ sub, role, id, exp }`.
3. React persists the token and the serialised user profile in local storage.
4. The request interceptor injects `Authorization: Bearer <token>` on every subsequent call.
5. The response interceptor treats any HTTP 401 as an expired session, purges local storage and redirects to the login route without throwing an uncaught UI error.

```
api.interceptors.request.use((config) => {
  if (isPublicAuthRequest(config.url)) return config;   // no token on auth forms
  const token = getStoredToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (response) => response,
  (error) => {
    const isAuthForm = isPublicAuthRequest(error.config?.url);
    if (error.response?.status === 401 && !isAuthForm) {
      clearAuthSession();
      window.dispatchEvent(new Event('pulmo-ai:session-expired'));
    }
    return Promise.reject(error);
  }
);
```

A 401 does not navigate by assigning window.location. It clears the stored session and dispatches a custom DOM event, which AuthContext listens for and turns into a React state update. That keeps navigation inside the router, so the redirect is a normal client-side transition rather than a full document reload, and the login page renders immediately instead of after a round trip. The same isPublicAuthRequest guard suppresses both the token injection and the expiry handling for the login, register and password routes.

Because a 401 interceptor performs a hard navigation, genuine credential rejections on the login form are returned as HTTP 400 with a field-level message, while HTTP 401 is reserved for expired or invalid tokens. This separation prevents a mistyped password from being misread as a session expiry.

## 3.3 Binary Multipart Streaming & the Boundary Trap

Radiograph screening and avatar uploads bypass JSON. The client builds a `FormData` object and lets the browser set the `Content-Type` header, including its generated multipart boundary. FastAPI streams the bytes to disk through `UploadFile = File(...)` without buffering the whole image in memory.

**Defect class documented from production.** Manually assigning `"Content-Type": "multipart/form-data"` in a request interceptor strips the boundary token, and the server then rejects the payload with `400 Missing boundary in multipart`. The symptom surfaces in the browser console as an opaque CORS complaint, which misdirects diagnosis. The correct rule is to never set the header manually for `FormData`; let the runtime supply it. All five multipart call sites in this codebase now omit the header.

## 3.4 Real-Time WebSocket Engine

To avoid polling, a WebSocket channel is maintained by a connection manager in `backend/app/api/websockets.py` on `/api/ws/queue`. Because the socket has no request-scoped database dependency, the session factory is published on application state at startup. Connected client screens receive queue events in roughly ten milliseconds and re-fetch authoritative state.

## 3.5 CORS Configuration

Cross-Origin Resource Sharing is configured through FastAPI's CORSMiddleware with an explicit allow-list rather than a wildcard, because credentialed requests cannot be combined with a wildcard origin. Seven development origins are permitted, covering the Vite default port, the ports this project actually occupied when 5173 was taken, their loopback aliases, and the common React dev port:

```
allow_origins=[
  "http://localhost:5173", "http://localhost:5174", "http://localhost:5175",
  "http://127.0.0.1:5173", "http://127.0.0.1:5174", "http://127.0.0.1:5175",
  "http://localhost:3000",
]
allow_credentials=True
allow_methods=["*"]
allow_headers=["*"]
```

**Correction against the previous issue.** The prior revision stated that static upload directories were mounted through a custom CORSStaticFiles subclass that forcibly appended `Access-Control-Allow-Origin: *`. That is no longer the case and the behaviour was unsafe. Only the low-sensitivity profile avatar directory is mounted as a public static mount. Diagnostic radiographs are deliberately excluded from static mounting.

## 3.6 Authenticated Medical Media Serving

Diagnostic images require authorisation, but an HTML `<img>` element cannot attach an `Authorization` header. The system therefore accepts the bearer token as a query parameter on a dedicated route and re-verifies record ownership on every single request:

```
GET /api/media/radiology/{filename}?token=<jwt>

  1. Reject absent token                      -> 401
  2. Reject non-basename / bad extension      -> 400
  3. Resolve the owning Prediction row by stem
  4. Enforce _can_access_prediction()         -> 404 if not permitted
  5. Confirm the resolved path is inside uploads/ and exists -> 404
  6. Stream the file with a derived media type
```

**Filename-stem resolution.** The Grad-CAM overlay is written as a sibling of the original image using the suffix `_gradcam.jpg` and is never referenced by a database column. The route therefore strips that suffix to locate the owning record, then serves the requested variant. Resolving strictly by `image_path LIKE '%filename'` made the explainability layer permanently unreachable, returning 404 for every heatmap request. Regression coverage for both the original and the overlay is provided in `tests/test_media_route.py`.

# 4. Deep Learning Model Pipeline & the Six-Stage Computer Vision Sanity Gate

## 4.1 Network Architecture

The diagnostic backbone is **DenseNet-121**, instantiated from torchvision and retargeted to the project's three-class problem set by replacing the final fully connected layer:

```
model = densenet121(weights=None)
num_ftrs = model.classifier.in_features
model.classifier = nn.Linear(num_ftrs, 3)
  # 3 classes: Normal, Pneumonia, TB
model.load_state_dict(torch.load(MODEL_PATH, map_location='cpu'))
model.eval()
```

DenseNet-121 was selected for its dense concatenation architecture: every convolutional block receives the feature maps of all preceding blocks, which mitigates vanishing-gradient degradation and permits feature reuse without additional parameters. For radiographs this matters, because consolidation and cavitation are often faint, low-contrast, spatially distributed findings that benefit from deep feature reuse. Weights load onto the CPU so the service runs on hardware without a CUDA runtime. The checkpoint `app/ml_model/chest_xray_densenet121.pth` is 28,433,177 bytes and is verified present in the deployment.

## 4.2 Preprocessing Contract

Inference-time preprocessing must match training exactly. A deterministic resize, ImageNet normalisation and tensor conversion pipeline is defined once and reused for both the prediction path and the Grad-CAM path:

```
transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])
```

**Design decision.** No random rotation, flip, colour jitter or RandomResizedCrop is applied at inference. Augmentation belongs to the offline training pipeline only, and applying a stochastic transform to a diagnostic submission would make the reported confidence non-reproducible for the same uploaded file. The repository ships the inference path only; training scripts are not part of the delivered tree, which is recorded as a limitation in Section 15.

## 4.3 Prediction & Class Semantics

| Index | Label | Clinical Interpretation |
|---|---|---|
| 0 | Normal | No radiographic evidence of active pulmonary abnormality within model sensitivity. |
| 1 | Pneumonia | Findings consistent with airspace consolidation. |
| 2 | TB | Findings consistent with tuberculous involvement, classically apical. |

A forward pass produces three logits, which are converted to a calibrated probability distribution with a softmax, and the arg-max index selects the reported class. The full distribution is persisted alongside the winner so historical records remain auditable and re-renderable:

```
probs     = torch.nn.functional.softmax(output, dim=1)[0].tolist()
all_scores = {labels[i]: round(probs[i], 4) for i in range(len(labels))}
final_result   = labels[final_index]
confidence     = all_scores[final_result]
return final_result, round(confidence, 4), all_scores, heatmap_path
```

**Upload admission control.** The route rejects any filename whose extension is not jpg, jpeg or png, rejects empty uploads, and caps the payload at 10 MB with HTTP 413. Accepted files are stored under a freshly generated UUID filename, which removes any attacker-supplied path component. Two honest caveats belong on record: the type check inspects only the extension rather than the file's magic bytes, and the size check runs after the upload has been read into memory, so the ceiling bounds disk usage but not peak memory. Both are listed in Section 15.

**Clinical safety invariant.** predict_image() raises immediately when the model is not connected, and the route converts that into HTTP 503. The code comment states the rule explicitly: random predictions are restricted and cannot be performed. A screening tool that silently invented a plausible label when its weights failed to load would be indistinguishable from a working one at the interface, which is unacceptable in a diagnostic context.

## 4.4 Grad-CAM Explainability

A classifier that outputs only a label and a probability is not clinically actionable. The system therefore computes a Grad-CAM overlay for every accepted prediction. The activation gradients of the final convolutional block are averaged over spatial dimensions and reweighted against the target-class activation map, producing a coarse localisation heatmap:

```
1. Forward pass, retaining the last convolutional feature map A (N, C, H, W)
2. Backward pass to the same block to obtain dY/dA
3. weights = mean(dY/dA, axis=(2, 3))          # per-channel importance
4. cam     = ReLU(sum_c weights_c * A_c)         # (N, H, W)
5. Normalise cam to [0, 1], upscale to source resolution
6. OpenCV JET colormap -> BGR overlay -> composite over the radiograph
7. Persist as <original_stem>_gradcam.jpg beside the original
```

Interpretation for the two positive classes: TB findings are expected to concentrate in the apical and upper-zone lung fields where cavitation classically develops, whereas pneumonia findings are expected as patchy, asymmetric consolidation, typically lobar. The heatmap is a model-attribution aid for the reviewing clinician. It is not an independent second opinion and must not be presented as one.

## 4.5 The Six-Stage Computer Vision Sanity Gate

Before any tensor reaches the network, the candidate image must survive six independent OpenCV stages implemented in `app/services/image_validator.py`. The function returns `(None, None)` to accept, or a human-readable reason string to reject with HTTP 400. Cascade classifiers are preloaded once at module import so the gate adds no per-request model load.

| # | Stage | Technique | Rejects |
|---|---|---|---|
| 1 | Dimension & aspect ratio | Minimum and maximum pixel bounds plus a maximum width:height deviation | Thumbnails, icons, screenshots, banners, extreme panoramic strips |
| 2 | Focus / blurriness | Variance of the Laplacian kernel on a greyscale copy | Out-of-focus captures, motion-blurred photographs |
| 3 | Dynamic range & contrast | Histogram spread and radiographic contrast statistics | Flat grey uploads, blank film, fully saturated frames |
| 4 | Chrominance & skin | Saturation analysis in HSV plus a skin-chrominance mask in YCrCb and HSV | Colour photographs, scanned film with a heavy uniform tint, objects with varied colour |
| 5 | Face & biometric cascade | Viola-Jones face detection, refined by eye detection inside the face region, plus checks for facial colour and prominence | Selfies, portraits, photographs of people, faces pasted onto images |
| 6 | Structural edge verification | Edge-density and structural analysis across the thoracic field for rib, spine and clavicle signature edges | Natural scenes and non-anatomical images that survive stages 1 to 5 |

**Correction against the previous issue.** The prior revision documented only two checks: a monochromatic test and a Laplacian blurriness test. The deployed gate has six stages, including a biometric face and eye cascade and a structural edge verification pass, and the documentation now reflects the implementation.

Defence in depth: the gate is an ergonomic and safety filter, not a security control. It reduces wasted inference and prevents obvious misclassification of unrelated imagery, but it is a heuristic and can be evaded. The hard guarantee against fabricated output is the model-connection invariant in Section 4.3.

# 5. Relational Database Design, Schema Reference & Cascading Integrity

The schema declares seven tables. Foreign keys, uniqueness constraints, check constraints and explicit cascade rules protect referential integrity across the clinical record graph.

## 5.1 users - Multi-role identity table

A single table backs all three roles, discriminated by the `role` column (patient, doctor, admin). Core identity columns `phone` and `cnic` are each uniquely indexed, because the login form accepts either identifier. The doctors-only fields (`specialization, clinic_name, address, about, timetable`) and the clinic-operations fields (`token_mode, max_tokens, consultation_fee`) are nullable and ignored for patients.

**Queue economics are snapshotted at booking time.** `consultation_fee` on the doctor row and `fee_charged` on the appointment row are separate columns. The fee is copied onto the appointment when it is issued, so a later price change by the doctor never retroactively alters what an existing patient was quoted. `token_mode` is either `fixed` (slots start at 1 and increment) or `dynamic` (slots start at 100 and decrement, so the lowest number always denotes the current patient).

## 5.2 Schema column reference

| Table | Key Columns | Purpose & Integrity Rules |
|---|---|---|
| users | id PK, name, phone UQ, cnic UQ, hashed_password, role, profile_pic, specialization, clinic_name, address, gender, age, token_mode, max_tokens, about, timetable, consultation_fee, used_access_code UQ, created_at | Unified identity for all roles. used_access_code is unique so a doctor access code cannot be redeemed twice, even across a restart. |
| doctor_patients | id PK, doctor_id FK, patient_id FK | Explicit many-to-many link so a patient may be treated by several doctors, and a doctor may follow several patients. |
| predictions | id PK, user_id FK, doctor_id FK (nullable), image_path, result, confidence, all_scores, created_at | Immutable diagnostic record. Nullable doctor_id supports a screening result created without a treating physician. all_scores retains the full probability vector. |
| appointments | id PK, doctor_id FK, patient_id FK, booking_date IDX, token_number, status, fee_charged, patient_name_override, created_at | The OPD queue. status is pending, serving or completed. patient_name_override lets a doctor issue a token to a family member or offline walk-in who has no account. |
| registration_requests | id PK, doctor_id FK, name, phone IDX, cnic IDX, gender, age, booking_date, token_number, status, rejection_reason, created_at | Guest registration lifecycle. status is pending, approved or rejected; a populated rejection_reason accompanies a rejection. |
| site_settings | id PK, setting_key UQ, setting_value | Global key/value configuration. |
| contact_messages | id PK, name, email, whatsapp, subject, message, is_read, doctor_id FK (nullable), created_at | Public contact form submissions, optionally attributed to a doctor. is_read is an integer boolean. |

# 6. File-by-File Repository Breakdown & Architectural Map

Line counts and byte sizes below are read directly from the working tree. Compiled __pycache__ artefacts and the model checkpoint are excluded from the source metrics. Totals: **30 frontend JavaScript and JSX modules, 17,083 lines**, plus a 307-line stylesheet; **14 backend modules, 3,689 lines**; **6 test modules, 1,127 lines**.

## *6.1 Frontend - src/ (17,083 lines)*

| Path | Lines | Architectural Responsibility |
|---|---|---|
| pages/DoctorDashboard.jsx | 1,796 | Doctor cockpit: live call-next panel, per-patient token issue, guest approval, fee collection. |
| pages/Patients.jsx | 1,888 | Patient registry with search, filtering, vitals and history drill-down. |
| pages/Profile.jsx | 1,505 | User profile editor, avatar upload, password and preference management. |
| pages/BookAppointment.jsx | 1,421 | Doctor discovery and booking. Directory renders a two-column mobile / three-column desktop grid with a full-width search filter. |
| pages/History.jsx | 1,594 | Past prediction archive with per-class filters, clinical PDF report generation, radiograph download, and permanent record deletion. |
| pages/AdminDashboard.jsx | 1,362 | Administrative CMS: user management, settings, contact inbox. |
| pages/GuestBooking.jsx | 968 | Unauthenticated walk-in registration and token request lifecycle. |
| pages/Contact.jsx | 591 | Public contact form and doctor messaging entry point. |
| pages/Predict.jsx | 545 | Radiograph upload workspace: drag-and-drop, sample selection, CV-gate feedback, result rendering, Grad-CAM display, model-status gating. |
| pages/Login.jsx | 512 | Role-aware sign-in accepting phone or CNIC, with three-second alert auto-dismiss. |
| pages/ForgotPassword.jsx | 434 | Credential recovery guidance flow. |
| pages/TodayTokens.jsx | 441 | Date-scoped token ledger. |
| pages/About.jsx | 370 | Project narrative, technology showcase, and stacked doctor call-to-actions. |
| pages/Terms.jsx | 237 | Sixteen-section terms of service with Lucide iconography. |
| pages/Privacy.jsx | 187 | Sixteen-section privacy policy with Lucide iconography. |
| pages/Payments.jsx | 226 | Fee summary and payment records. |
| components/Navbar.jsx | 531 | Role-aware navigation shell, light/dark toggle, and brand mark rendering in desktop and mobile layouts. |
| components/Footer.jsx | 308 | Site footer with legal and contact links. |
| components/DoctorContactModal.jsx | 279 | Direct doctor messaging modal. |
| components/ResultCard.jsx | 279 | Prediction presentation with class, confidence, full distribution, Grad-CAM toggle, and non-wrapping report, download and delete actions. |
| components/UploadBox.jsx | 182 | Drag-and-drop upload control with client-side pre-checks. |
| components/PasswordStrengthIndicator.jsx | 173 | Live password entropy feedback. |
| services/api.js | 184 | Axios instance, JWT interceptors, and media URL construction. |
| services/AuthContext.jsx | 66 | Authentication state provider and role propagation. |
| services/ThemeContext.jsx | 38 | Light and dark theme state. |
| utils/alert.js | 35 | SweetAlert2 wrapper for uniform clinical confirmations. |
| App.jsx | 148 | Router tree, role guards, and lazy route registration. |
| index.css | 307 | Tailwind layers, glassmorphism utilities, and design tokens. |
| main.jsx | 16 | ReactDOM root bootstrap. |

## *6.2 Backend - app/ (3,689 lines, 14 modules)*

| Path | Lines | Architectural Responsibility |
|---|---|---|
| api/routes.py | 2,220 | Primary router: 39 endpoints covering auth, prediction, media, patients, appointments, guest registration, messaging and deletion. |
| api/admin.py | 479 | Administrative router: 14 endpoints for users, settings, contact messages, statistics and clinical oversight of predictions and appointments. |
| api/auth_utils.py | 46 | bcrypt hashing, JWT issuance, and the OAuth2 bearer dependency. |
| api/websockets.py | 45 | Connection manager and /api/ws/queue broadcast endpoint. |
| core/config.py | 42 | Typed settings with production fail-fast guards on secrets and keys. |
| db/models.py | 129 | Seven SQLAlchemy table definitions with relationships and constraints. |
| db/database.py | 16 | Engine, session factory, and Base declarative registry. |
| main.py | 64 | Application assembly, middleware, static mounts, and model preload. |
| schemas/request.py | 10 | Shared Pydantic request models. |
| models/densenet.py | 24 | DenseNet-121 factory retargeted to three output classes. |
| services/predict.py | 200 | Model lifecycle, preprocessing, inference, and Grad-CAM generation. |
| services/image_validator.py | 130 | The six-stage OpenCV radiograph sanity gate. |
| services/queue.py | 271 | Token allocation, serving state machine, and queue-window enforcement. |
| utils/preprocess.py | 13 | Shared image decoding and resize helpers. |

### *6.3 Model checkpoint, tests and deployment*

| Path | Size | Notes |
|---|---|---|
| app/ml_model/chest_xray_densenet121.pth | 28,433,177 b | Trained DenseNet-121 checkpoint, loaded onto CPU. |
| tests/conftest.py | 68 lines | Isolated SQLite fixture with StaticPool and upload-directory redirection. |
| tests/test_auth_security.py | 430 lines | Authentication, role, registration-code, prediction-ownership and public-queue privacy coverage. |
| tests/test_media_route.py | 148 lines | Media authorization, Grad-CAM resolution, traversal and delete cleanup. |
| tests/test_queue.py | 141 lines | Token allocation and queue-window behaviour. |
| tests/test_admin_user_integrity.py | 335 lines | CNIC/phone cross-column uniqueness, avatar-safe user deletion, contact-settings validation and admin clinical-oversight authorisation. |
| run.py | - | Development entry point; serves app.main:app on 0.0.0.0:8000 with reload. |
| docker/docker-compose.yml | - | Two-service Compose stack (backend, frontend/Nginx). |
| docker/backend.Dockerfile | - | Backend container image definition. |
| docker/frontend.Dockerfile | - | Multi-stage SPA build served by Nginx. |
| docker/nginx.conf | - | Reverse proxy, TLS termination, and SPA history fallback. |

**Correction against the previous issue.** The prior file map omitted `Navbar.jsx`, `Footer.jsx`, `DoctorContactModal.jsx`, `PasswordStrengthIndicator.jsx`, `ThemeContext.jsx`, `image_validator.py`, `queue.py`, `preprocess.py`, `densenet.py`, `schemas/request.py` and the entire test suite, and it understated the endpoint surface as "over 30" when the codebase declares 51. Every size figure in the previous issue was also stale.

# 7. Comprehensive Frontend Page-by-Page Breakdown

## 7.1 Public and marketing surface

| Page | Lines | Function |
|---|---|---|
| Home | 110 | Hero section, statistics, disease education cards, and a call-to-action that routes to **Doctor Registration** rather than a generic sign-up. |
| About | 370 | Project narrative, architecture highlights, and stacked primary and secondary doctor call-to-action buttons. |
| Contact | 586 | Public enquiry form, doctor messaging, and clinic location details. |
| Terms | 237 | Sixteen sections covering acceptable use, the advisory nature of results, liability limits, data handling, and intellectual property. |
| Privacy | 187 | Sixteen sections covering data categories, processing purposes, retention, disclosure, and user rights. |

**Scope note on legal content.** The Terms and Privacy pages state deployment and data-handling commitments such as transport encryption, storage jurisdiction, and human review of results. These are policy commitments of the deploying institution, not properties the source code can prove. They require review by the supervising faculty and the institution before publication, and should not be cited in this report as verified technical controls.

## 7.2 Authentication pages

| Page | Lines | Function |
|---|---|---|
| Login | 512 | Role-aware sign-in. Accepts phone or CNIC. Surfaces a server-provided error and auto-dismisses the alert after three seconds so the user is returned to the form. |
| Register | 657 | **Doctor-only self-registration.** Collects professional identity, clinic details, specialisation, address, fee, token policy, timetable and a one-time doctor access code. Public patients cannot self-register here. |
| ForgotPassword | 434 | Credential recovery guidance. |

**Correction against the previous issue.** The previous revision described `Register.jsx` as a dual-role patient and doctor form in which patients entered a name, phone and CNIC. That workflow has been withdrawn. Public registration is restricted to verified doctors holding a valid access code, and patient accounts are provisioned by a verified doctor or through the guest-token flow in Section 8.3.

## 7.3 Clinical application pages

| Page | Lines | Function |
|---|---|---|
| Predict | 545 | Upload workspace. Presents the six-stage validation outcome, the predicted class with confidence and full distribution, the Grad-CAM overlay, and clinical disclaimers. Submission is disabled while the model reports disconnected. |
| History | 1,594 | Filterable prediction archive, A4 report generation via html2pdf.js, radiograph download, and permanent deletion of a record and its overlay. |
| BookAppointment | 1,421 | Doctor directory, availability preview, and booking confirmation. |
| GuestBooking | 968 | Walk-in registration, token request, and status tracking. |
| Patients | 1,712 | Doctor-side patient registry. |
| Profile | 1,505 | Account and clinic settings, avatar upload. |
| TodayTokens | 460 | Date-scoped token ledger. |
| Payments | 226 | Fee records. |

## 7.4 Dashboards

| Page | Lines | Function |
|---|---|---|
| DoctorDashboard | 1,832 | The operational centrepiece: live serving patient, queue management, guest approval, and fee collection. |
| AdminDashboard | 1,130 | User administration, global settings, contact inbox, platform statistics. |

# 8. Role-Based Portals & Functional Workflows

## 8.1 Access model

A single `users` table underpins three portals. Public registration creates **doctors only**. Patients are provisioned in one of two ways: a verified doctor registers them from the doctor portal, or an unauthenticated visitor books a walk-in token and is approved by a doctor. This inverts the previous design and removes unverified patient sign-up.

| Portal | Entry condition | Capabilities |
|---|---|---|
| Patient | Provisioned account, or guest token approval | Submit radiographs, view and download own results and overlays, book appointments, manage profile. |
| Doctor | Registration with a valid single-use access code | Verify the CV gate, review predictions and Grad-CAM, issue and call tokens, register patients, approve guest requests, collect fees, message patients. |
| Admin | Seeded credential | Manage users and roles, global settings, contact inbox, platform statistics, audit oversight. |
| Guest | None (unauthenticated) | Request a walk-in token, then poll status until a doctor approves or rejects. |

## 8.2 Doctor registration with single-use access codes

1. The applicant submits professional details together with a doctor access code.
2. The server rejects any request whose `role` is not `doctor` with HTTP 403, so the endpoint cannot be repurposed to create patient accounts.
3. The code is parsed from the comma-separated `REGISTRATION_KEYS` setting and compared exactly; an unmatched code returns HTTP 403 `Invalid Doctor Access Code`.
4. A uniqueness probe rejects the code if any user already holds it in `used_access_code`.
5. The code is recorded on the new user row. Because the column is uniquely indexed, replay is rejected at the database layer even if the in-process burn set is lost on restart.

The implementation burns the code in a process-local set for defence in speed, and relies on the unique index for correctness. Section 15 records the multi-instance gap this leaves.

## 8.3 Patient provisioning workflows

### *8.3.1 Doctor-registered patient*

A signed-in doctor submits the patient's demographics from the Patients portal. The server validates the payload, creates a user row with role `patient`, and inserts a `doctor_patients` link so the relationship survives the absence of any invitation token. The patient's password is set by the doctor.

### *8.3.2 Guest walk-in token*

A visitor with no account supplies a name, phone, CNIC, gender, age and intended booking date. The server creates a `registration_requests` row in state `pending` and returns a reference. A doctor reviews pending requests in the dashboard and either approves, which allocates a real token number in the appointment table, or rejects with a stored reason. The guest polls or watches for the state change.

## 8.4 The OPD token queue state machine

Queue logic is centralised in `app/services/queue.py` so that token allocation semantics are identical no matter which portal triggers them.

```
[ pending ] --doctor calls--> [ serving ] --complete--> [ completed ]
     |                              |
     +------ cancel ----------------+---- cancel --> (removed)
```

| Rule | Behaviour |
|---|---|
| Token mode: fixed | Numbers allocate upward from 1, so display order matches arrival order. |
| Token mode: dynamic | Numbers allocate downward from a high base so the smallest number is always the current serving patient. |
| Concurrency | Allocation re-checks the highest issued token for the doctor and date inside a transaction, so simultaneous bookings cannot collide on a number. |

| Rule | Behaviour |
|---|---|
| Queue window | A doctor-configured maximum caps the number of open tokens; a full queue rejects new bookings with a clear message. |
| Fee snapshot | The doctor's consultation fee is copied onto the appointment at issue time. |
| Offline patients | `patient_name_override` allows a token to be issued to a family member or walk-in who has no account. |
| Propagation | Every state change broadcasts over `/api/ws/queue` so all connected displays update without polling. |

## 8.5 Doctor-patient linking

The `doctor_patients` junction table models a genuine many-to-many relationship. A patient may be under the care of several clinicians, and a doctor may follow several patients. This supports continuity of care and lets either party unlink, which the API exposes as an explicit unlink operation.

# 9. Complete RESTful API & WebSocket Endpoint Directory

The service declares **55 route handlers**: 39 in `api/routes.py` (38 REST plus the queue WebSocket), 14 in `api/admin.py`, and the unauthenticated service root in `main.py`. That is 54 REST handlers and one WebSocket, and because a duplicated declaration was removed during this review the declared and reachable surfaces are the same **55**. Every entry below is transcribed from the source decorators, including their line numbers.

### 9.1 Authentication and identity (routes.py)

| Method | Path | Line | Access | Purpose |
|---|---|---|---|---|
| POST | /api/auth/register | 270 | Public, doctor only | Doctor self-registration; requires a valid unused access code. |
| POST | /api/auth/login | 364 | Public | Phone or CNIC plus password; returns a JWT and the user profile. |
| POST | /api/auth/update-profile | 420 | Bearer | Update the signed-in user's own profile. |
| POST | /api/auth/reset-password | 525 | Bearer | Password change for the authenticated account. |
| GET | /api/doctors/list | 1139 | Public | Public doctor directory used by the booking portal. |

### 9.2 Inference and medical media (routes.py)

| Method | Path | Line | Access | Purpose |
|---|---|---|---|---|
| GET | /api/model-status | 696 | Public | Reports model connectivity and the active error state. Drives frontend submission gating. |
| POST | /api/predict | 566 | Bearer | Multipart radiograph upload. Runs the six-stage CV gate, then DenseNet inference and Grad-CAM. Returns HTTP 503 if the model is disconnected. |
| GET | /api/media/radiology/{filename} | 224 | Bearer or ?token | Authenticated radiograph streaming with per-request ownership verification. Resolves both the original and the _gradcam overlay. |
| GET | /api/patient/history | 703 | Bearer | Patient's own prediction archive with filtering. |

### 9.3 Doctor practice management (routes.py)

| Method | Path | Line | Access | Purpose |
|---|---|---|---|---|
| GET | /api/doctor/dashboard | 1020 | Doctor | Cockpit aggregate: queue, earnings and today's statistics. |
| GET | /api/doctor/search-patient | 775 | Doctor | Look up a patient by phone or CNIC. |
| POST | /api/doctor/register-patient | 812 | Doctor | Provision a patient account and create the care link. |
| PUT | /api/doctor/update-patient/{patient_id} | 945 | Doctor | Amend a registered patient's demographics. |
| DELETE | /api/doctor/unlink-patient/{patient_id} | 927 | Doctor | Dissolve the care relationship. |
| POST | /api/doctor/link-patient | 904 | Doctor | Attach an existing account to the doctor's panel. |
| POST | /api/doctor/link-prediction-to-patient | 186 | Doctor | Attribute an unowned screening result to a patient record. This handler was previously declared twice; the duplicate has been removed. See Section 9.11. |
| DELETE | /api/doctor/delete-prediction/{prediction_id} | 152 | Owner or treating doctor | Permanently delete the record, the radiograph and the paired Grad-CAM overlay. |
| GET | /api/doctor/payments | 2170 | Doctor | Fee and payment records for the practice. |

### 9.4 Appointments and the token queue (routes.py)

| Method | Path | Line | Access | Purpose |
|---|---|---|---|---|
| GET | /api/appointments/queue-window | 1191 | Bearer | Remaining capacity in the active queue window. |
| POST | /api/appointments/book | 1211 | Bearer | Allocate a token and create the appointment. |
| GET | /api/appointments/booked-tokens | 1158 | Bearer | Tokens already issued for the doctor and date. |
| GET | /api/appointments/patient-bookings | 1317 | Bearer | Current patient's own bookings. |
| GET | /api/appointments/doctor-queue-public | 1361 | Public | Read-only queue projection for a waiting-room display. |

| Method | Path | Line | Access | Purpose |
|---|---|---|---|---|
| GET | /api/appointments/doctor-today | 1401 | Doctor | Date-scoped ledger for the doctor. |
| POST | /api/appointments/move-to-tomorrow | 1446 | Doctor | Rollover of unfinished appointments to the next day. |
| POST | /api/appointments/update-status | 1513 | Bearer | Apply a status transition to the caller's own appointment. |
| POST | /api/appointments/check-in/{appointment_id} | 1592 | Bearer | Move a pending appointment to serving and broadcast the change. |
| GET | /api/appointments/serving-status | 1637 | Public | Status of the current serving patient. |
| PUT | /api/appointments/update/{appointment_id} | 1725 | Bearer | Amend appointment details. |
| DELETE | /api/appointments/cancel/{appointment_id} | 1671 | Bearer | Cancel an appointment and release its token. |

### 9.5 Guest registration lifecycle (routes.py)

| Method | Path | Line | Access | Purpose |
|---|---|---|---|---|
| POST | /api/registration/request | 1852 | Public | Submit a walk-in registration request; returns pending. |
| GET | /api/registration/status/{phone_or_cnic} | 1948 | Public | Poll the approval state using the applicant's identifier. |
| GET | /api/registration/requests | 1966 | Doctor | List pending guest requests. |
| POST | /api/registration/{request_id}/approve | 2004 | Doctor | Approve a request and allocate a token. |
| POST | /api/registration/{request_id}/reject | 2127 | Doctor | Reject a request with a stored reason. |

### 9.6 Messaging and contact (routes.py)

| Method | Path | Line | Access | Purpose |
|---|---|---|---|---|
| POST | /api/contact/submit | 105 | Public | Public enquiry submission, optionally attributed to a doctor. |
| GET | /api/doctor/messages | 119 | Doctor | Messages addressed to the signed-in doctor. |
| PUT | /api/doctor/messages/{msg_id}/read | 128 | Doctor | Mark a message as read. |
| DELETE | /api/doctor/messages/{msg_id} | 140 | Doctor | Remove a message from the doctor's inbox. |

### 9.7 Administrative router (admin.py, 14 endpoints)

| Method | Path | Line | Access | Purpose |
|---|---|---|---|---|
| GET | /api/admin/users | 196 | Admin | List all users for moderation. |
| POST | /api/admin/users/{user_id}/role | 217 | Admin | Change a user's role. |
| PUT | /api/admin/users/{user_id}/details | 235 | Admin | Amend a user record. |
| DELETE | /api/admin/users/{user_id} | 269 | Admin | Remove a user account. |
| GET | /api/admin/messages | 155 | Admin | All contact submissions. |
| PUT | /api/admin/messages/{msg_id}/read | 160 | Admin | Mark a submission as read. |
| DELETE | /api/admin/messages/{msg_id} | 169 | Admin | Delete a submission. |
| GET | /api/admin/settings/contact | 122 | Admin | Read contact configuration. |
| PUT | /api/admin/settings/contact | 136 | Admin | Update contact configuration. |
| GET | /api/admin/stats | 178 | Admin | Platform statistics for the dashboard. |
| GET | /api/admin/predictions | 348 | Admin | List predictions with patient names, paginated. |
| DELETE | /api/admin/predictions/{prediction_id} | 392 | Admin | Delete a prediction, its radiograph and its Grad-CAM overlay. |
| GET | /api/admin/appointments | 412 | Admin | List appointments with patient names, paginated. |
| DELETE | /api/admin/appointments/{appointment_id} | 462 | Admin | Delete a booking. |

### 9.8 WebSocket channel

| Channel | Line | Protocol | Purpose |
|---|---|---|---|
| /api/ws/queue | 1814 | WebSocket | Bidirectional queue channel. The server broadcasts token state changes; clients re-fetch authoritative state on receipt rather than treating the payload as canonical. |

### 9.9 Endpoint totals

| Group | Count |
|---|---|
| Authentication and identity (Section 9.1) | 5 |
| Inference, medical media and history (9.2) | 4 |
| Doctor practice management (9.3) | 9 |
| Appointments and the token queue (9.4) | 12 |
| Guest registration lifecycle (9.5) | 5 |
| Messaging and contact (9.6) | 4 |
| Administrative router (9.7) | 14 |
| WebSocket channel (9.8) | 1 |
| Service root (main.py) | 1 |
| REST subtotal | 54 |
| Total handlers declared and reachable | 55 (54 REST + 1 WebSocket) |

The counts above were reconciled against the FastAPI route table at generation time: 54 `APIRoute` objects plus one `APIWebSocketRoute`, with no duplicate method and path pair remaining.

### 9.10 Public surface and exposure review

Thirteen endpoints under `/api` are reachable without a bearer token, plus the unauthenticated service root, giving an unauthenticated surface of **14 handlers** against **41 authenticated REST handlers**. This was established by probing the running service with no Authorization header and recording the response code, not by reading decorators: this codebase validates tokens inside handler bodies through `get_current_user_from_token` rather than through injected dependencies, so dependency introspection alone does not reveal the real access model. The table lists all thirteen.

| Public endpoint | Why it is public | Residual exposure |
|---|---|---|
| GET /api/model-status | Drives frontend gating and is diagnostic only. | Leaks whether the model loaded and the error text. |
| POST /api/auth/login | Authentication entry point. | Subject to brute force; needs proxy rate limiting. |
| POST /api/auth/register | Doctor onboarding. | Gated by access codes, but codes are guessable if weak. |
| POST /api/auth/reset-password | Credential recovery. | Unauthenticated write path; needs rate limiting to prevent abuse. |
| POST /api/contact/submit | Public enquiry form. | Unauthenticated write path; vulnerable to spam without rate limiting. |
| POST /api/registration/request | Walk-in registration. | Same spam exposure as the contact form. |
| GET /api/registration/status/{id} | Guest status polling. | Enumerable; reveals whether an identifier has a pending request. |
| GET /api/doctors/list | Public directory. | Intentional; reveals clinician details intentionally published. |
| GET /api/appointments/doctor-queue-public | Waiting-room display. | Token numbers and status only. Patient names are reduced to initials and the fee is withheld, because this route is unauthenticated and previously returned both in full. |
| GET /api/appointments/serving-status | Queue display. | Publishes the token number currently being served. |
| GET /api/appointments/queue-window | Queue display. | Publishes the active token window bounds. |
| GET /api/appointments/booked-tokens | Queue display. | Publishes the set of booked token numbers only; no patient identity. |
| GET /api/admin/settings/contact | Public contact page. | Read-only; reveals published contact details by design. The matching PUT is admin only. |

**Deliberately excluded from the public surface.** `GET /api/media/radiology/{filename}` is sometimes described as public because it also accepts the token as a query parameter, but an unauthenticated request returns 401, so it is counted among the 41 authenticated handlers. `POST /api/predict` likewise returns 401 without a token.

### *9.11 Defect found and resolved during this review: duplicated route*

`POST /api/doctor/link-prediction-to-patient` was declared twice in `api/routes.py`, with an identical function name, signature and body. FastAPI matches routes in registration order, so the second declaration was unreachable dead code, and the two copies could silently diverge. This has been **resolved**: the redundant declaration was deleted and the surviving handler was kept, including its ownership check that restricts the operation to the doctor who owns the prediction. A regression test now asserts that a doctor cannot link another doctor's prediction.

**Correction against the previous issue.** The prior API table listed a different set of paths, including several that do not exist in the codebase, and omitted the administrative router, the guest registration lifecycle, the doctor practice endpoints and the WebSocket channel. This directory is now transcribed directly from the route decorators.

# 10. Step-by-Step Transaction Lifecycles

## 10.1 Radiograph screening request

1. The patient selects or captures a radiograph and submits it as multipart form data. The browser sets the multipart boundary automatically; the client never assigns the header.
2. The request interceptor attaches the bearer JWT.
3. The server enforces a 10 MB payload ceiling before the file is written to disk, so an oversized upload cannot exhaust storage.
4. `validate_chest_xray()` runs all six OpenCV stages. Any failure returns HTTP 400 naming the stage that rejected the image, and the frontend renders that reason so the user knows whether to retake the film.
5. The model-connection invariant is checked. A disconnected model returns HTTP 503 with an explicit message; no inference is attempted.
6. The image is normalised to a 224x224 tensor and passed through DenseNet-121 on the CPU.
7. Softmax yields the three-class distribution; the arg-max class and its probability are selected as the prediction.
8. Grad-CAM is computed for the winning class and written to `<stem>_gradcam.jpg` beside the original.
9. A `predictions` row stores the class, confidence, the full probability vector, the file paths and timestamps.
10. The response carries the result; the frontend renders the class, the confidence meter, the full distribution and the optional heatmap.
11. Both images are later retrievable only through the authenticated media route.

The live dataset currently holds three users (one administrator, one doctor and one patient), one doctor-patient link, three screening predictions, five site settings, and no appointments, guest registration requests or contact messages. The three predictions are synthetic rows created while exercising the upload, media and admin-oversight paths, so storage and integrity behaviour is documented from the schema and the automated suite rather than from genuine clinical volume.

## 10.2 OPD appointment and token lifecycle

1. The patient queries `/api/appointments/queue-window` and the booked-token list to learn what is still available.
2. The patient confirms, posting to `/api/appointments/book`. Inside a transaction the server re-checks capacity and allocates the next token according to the doctor's token mode.
3. The doctor's consultation fee is copied onto the appointment as `fee_charged`.
4. The appointment is created in `pending` state and the change is broadcast over `/api/ws/queue`.
5. At the clinic the doctor checks the patient in with `/api/appointments/check-in/{appointment_id}`, moving it to `serving`; the waiting room and the patient's handset update.
6. Unfinished appointments can be rolled forward with `/api/appointments/move-to-tomorrow`.
7. After consultation the status is advanced to `completed` and the fee captured.
8. Either party may cancel; the token is released and the queue is rebroadcast.

## 10.3 Guest walk-in registration

1. An unauthenticated visitor posts demographics and an intended date to `/api/registration/request`; a `registration_requests` row is created as `pending`.
2. The visitor polls `/api/registration/status/{phone_or_cnic}` or waits for the broadcast.
3. A doctor reviews the queue via `/api/registration/requests`.
4. Approval at `/api/registration/{request_id}/approve` allocates a real token number and creates the appointment.
5. Rejection at `/api/registration/{request_id}/reject` stores a reason the guest can read, keeping the audit trail on the record.

# 11. Deployment, Environment Configuration & Troubleshooting

## 11.1 Environment variables

Settings are typed through pydantic-settings in `app/core/config.py`. The service applies fail-fast guards in production rather than booting with insecure defaults:

```
if ENV == "production":
    if SECRET_KEY in ("your_super_secret_key", "changeme", ...):
        raise RuntimeError("SECRET_KEY must be overridden in production.")
    if len(REGISTRATION_KEYS.strip()) < 8:
        raise RuntimeError("REGISTRATION_KEYS must be set in production.")
```

| Variable | Purpose | Handling |
|---|---|---|
| DATABASE_URL | SQLAlchemy connection string | Secret. Never logged or documented. |
| SECRET_KEY | HS256 signing key | Secret. Validated at startup in production. |
| ALGORITHM | Token signing algorithm | Defaults to HS256. |
| ACCESS_TOKEN_EXPIRE_MINUTES | Token lifetime | Numeric; short-lived by design. |
| REGISTRATION_KEYS | Comma-separated doctor access codes | Secret. Rejects weak or unset values in production. |
| ENV | Deployment environment flag | Drives the production guards. |

This report intentionally omits all secret values. A real `.env` must never be committed to version control, and no database password, signing key or registration code may appear in documentation, screenshots or slide decks.

## 11.2 Running locally

```
# Backend
cd backend
pip install -r requirements.txt
python run.py                 # serves app.main:app on 0.0.0.0:8000

# Frontend (separate shell)
cd frontend
npm install
npm run dev                   # http://localhost:5173

# Quality gates
npm run lint                  # --max-warnings 0
npm run build
pytest                        # 61 tests
```

If port 5173 is occupied, Vite silently advances to 5174 or 5175. Because CORS permits the full set the application still works, but a hard-coded origin elsewhere will fail. In this project the dev server most often bound to **5174**.

## 11.3 Containers

The Compose stack lives in the `docker/` directory, not the repository root. It defines a backend service and a frontend service fronted by Nginx, which terminates TLS and falls back to index.html for client-side routes:

```
cd docker
docker compose up --build
```

## 11.4 Troubleshooting matrix

| Symptom | Root cause | Resolution |
|---|---|---|
| 400 Missing boundary in multipart | Content-Type was set manually, stripping the multipart boundary. | Delete the manual header from the Axios call and let the browser set it. |
| CORS error in console, upload fails | Usually this is the boundary defect above, not a CORS policy problem. | Inspect the request headers before touching allow_origins. |
| All Grad-CAM heatmaps return 404 | The media route resolved the overlay by LIKE match on the full filename and never matched. | Strip the _gradcam suffix, look up the owning prediction, then serve the requested variant. Covered by tests/test_media_route.py. |
| Prediction returns 503 | Model weights failed to load; random fallback is intentionally disabled. | Check /api/model-status and the startup log. Verify the 28 MB checkpoint is present. |

| Symptom | Root cause | Resolution |
|---|---|---|
| Radiograph loads as a broken icon | An <img> request was made without a token, or the record is not owned by the caller. | Append ?token= to the media URL, or use the api.js media URL helpers. |
| Doctor registration rejected | Missing, unknown or already-redeemed access code. | Verify REGISTRATION_KEYS and inspect used_access_code for prior redemption. |
| Blank page after a production deploy | Nginx lacks the SPA history fallback. | Ensure nginx.conf falls back to index.html for non-asset paths. |
| Frontend build fails on lint | ESLint runs with --max-warnings 0. | Fix the warning; the repository currently reports zero. |

# 12. Security Architecture & Access Control Model

## 12.1 Authentication and token lifecycle

Passwords are hashed with bcrypt using a per-password random salt, never a static salt or plaintext. Login returns an HS256-signed JWT carrying the subject, role, primary key and an expiry. Every protected route resolves the bearer token through a single dependency that rejects any signature mismatch or expiry with HTTP 401. The frontend distinguishes an expired session (401, which purges local state and redirects) from a credential rejection (400, shown inline), so a mistyped password is never misread as a session expiry.

## 12.2 Layered access control

| Layer | Control |
|---|---|
| Transport | CORS restricted to an explicit development allow-list rather than a wildcard, so credentialed requests remain valid. Production TLS terminates at Nginx. |
| Session | Stateless JWT verification on every request; a 401 triggers local state purge and redirect. |
| Routing | Frontend role guards prevent rendering of unauthorised portals, treated strictly as a user-experience measure. |
| Endpoint | Backend handlers re-derive the caller from the signed token and re-check role and record ownership per request. Client-side guards are never trusted. |
| Registration | Public sign-up is restricted to doctors holding an unredeemed access code, enforced server-side and backed by a unique index. |
| Media | Diagnostic radiographs are excluded from static mounting and streamed only through an authenticated route with per-request ownership verification. |
| Input validation | Pydantic schemas validate request bodies; the CV gate rejects non-radiographic uploads; a 10 MB ceiling bounds storage. |
| Fail-fast config | Production startup aborts on default secrets or weak registration keys. |
| Headers | Security response headers are applied by middleware to every response. |

## 12.3 Threats and controls

| Threat | Control | Residual risk |
|---|---|---|
| Credential stuffing | bcrypt cost, short token life, and registration codes for new doctors. | No application-layer rate limiting; it must be enforced at Nginx. |
| Broken object-level authorisation on media | Ownership is re-verified on every media request by database lookup. | Correctness depends on the ownership predicate; regression-tested. |
| Path traversal via crafted filenames | Basenames are enforced and the resolved path must remain inside the uploads directory. | None identified in the current implementation. |
| Fabricated diagnoses on model failure | HTTP 503 refusal; no random fallback path exists. | A disconnected model blocks screening entirely. This is the intended trade-off. |
| Access-code replay across instances | The unique index on used_access_code is authoritative; the in-process burn set is a fast path. | Redemption is not propagated across instances, so a race surfaces a duplicate-key error rather than a clean rejection. See Section 15. |
| Spam on public write endpoints | Registration codes gate doctor sign-up. | The contact form and walk-in request accept unauthenticated writes with no rate limit. |
| XSS or malicious uploads | React escapes rendered text by default; files are re-encoded by Pillow and served with derived media types. | No content-security-policy header is currently set. |
| Patient identity disclosure on the public waiting-room queue | The unauthenticated queue projection now returns the token number, the status and an initials-only patient label, and withholds the consultation fee. A regression test asserts that no full name and no fee field can appear in the response. | Initials remain re-identifiable in a small waiting room, so anyone who can read the display can still infer who is present. That is inherent to a public queue board. |
| Admins reading clinical records | The administrative list and delete endpoints are role-checked server-side on every request; the boundary was confirmed by probing with a doctor token and receiving 403. | Admins can see and delete predictions and appointments but cannot alter a diagnosis or a fee, and these are ordinary authenticated requests rather than separately audited ones. |

# 13. Automated Test Suite & Verification Evidence

The backend ships an automated suite of **61 passing tests** across five modules, 1,054 lines, supported by a further 73 lines of fixtures in conftest.py and __init__.py. Tests run against an isolated in-memory SQLite database with `StaticPool` so a single shared connection persists across the session, and the upload directory is redirected to a temporary path so tests never touch clinical files.

| Module | Lines | Coverage |
|---|---|---|
| test_auth_security.py | 430 | Registration role enforcement, access-code validation and replay rejection, login with phone and CNIC, wrong-password handling, JWT claim integrity, expiry, 401 behaviour for missing or malformed tokens, doctor-only enforcement across the dashboard and its actions, prediction-ownership checks, and the public-queue privacy assertions. |
| test_media_route.py | 148 | Original-image serving, Grad-CAM overlay resolution for both PNG and JPEG variants, cross-patient access denial, traversal rejection, missing-file handling, query-token authentication, and confirmation that deletion removes both files. |
| test_queue.py | 141 | Token allocation in fixed and dynamic modes, capacity and queue-window enforcement, and serving state transitions. |
| test_admin_user_integrity.py | 335 | Cross-column CNIC and phone uniqueness on admin edits, avatar-safe user deletion, contact-settings validation for email, phone and WhatsApp values including atomicity on failure, and role enforcement on the administrative clinical-oversight endpoints. |

### *13.1 Verified state at time of writing*

| Check | Method | Result |
|---|---|---|
| Backend suite | pytest | 61 passed |
| Frontend lint | npm run lint | Zero errors, zero warnings |
| Frontend build | npm run build | Production bundle emitted successfully |
| Model checkpoint | File inspection | Present, 28,433,177 bytes |
| Model health | GET /api/model-status | connected: true |
| Route inventory | Introspection of the FastAPI route table | 54 REST + 1 WebSocket, with no duplicate method and path pair |
| Unauthenticated surface | Live probe of all 54 REST handlers with no Authorization header | 14 handlers reachable unauthenticated; the other 41 return 401 |
| Doctor-only enforcement | Live probe with an admin token against the doctor dashboard, patient search and payments | 403 for admin, 200 for doctor |
| Public queue disclosure | Live probe of the unauthenticated waiting-room queue route | Initials-only patient label, no full name, no fee |

**Methodological note.** The Grad-CAM media tests were written before the fix and observed to fail against the original LIKE-match implementation, then pass once filename-stem resolution was applied. The suite therefore detects the regression class rather than merely confirming current behaviour.

### *13.2 Coverage gaps*

- No frontend component or end-to-end browser tests; the UI relies on manual verification and lint.
- The six-stage CV gate has no unit tests with positive and negative image fixtures, which is the highest-value area for future coverage.
- Queue tests use the SQLite fixture, so PostgreSQL-specific concurrency semantics are not exercised.
- No load or performance testing of inference latency or WebSocket fan-out.
- No test asserts the absence of duplicate route registration, so the class of defect described in Section 9.11 could recur without being detected.

# 14. Appendix A: Engineering Change Log

This revision documents engineering work completed against the codebase during the final review pass, including a security hardening pass, together with the observations that remain open at the time of writing.

### *14.1 Defects resolved*

| Area | Symptom | Resolution |
|---|---|---|
| Grad-CAM media serving | Every heatmap request returned 404. The overlay was stored as <stem>_gradcam.jpg and queried with a trailing LIKE match, so the suffix never matched and no owning record was found. | The route now derives the requested stem, detects the _gradcam variant, resolves the owning prediction, re-verifies access, then serves the requested file with a derived media type. PNG and JPEG overlays are both covered. |
| Prediction deletion | Deleting a record left the radiograph and its Grad-CAM overlay orphaned on disk, consuming storage and leaving unreferenced clinical images. | The delete endpoint now removes the record and both sibling files. The UI wording was also corrected from "Archive" to a permanent "Delete" so the action is not misrepresented as reversible. |
| Multipart uploads | Five call sites set Content-Type manually, stripping the multipart boundary. The server rejected them with 400 Missing boundary in multipart, which surfaced as a misleading CORS error. | Manual headers removed from Register.jsx, both Patients.jsx uploads, Profile.jsx and DoctorDashboard.jsx. |
| Predict page icons | The Sparkles icon failed to render because the import was missing. | Import added in Predict.jsx. |
| Application branding | The page requested /vite.svg, the default Vite starter icon, so the browser tab and bookmarks showed the framework logo instead of the project's mark. | Generated a multi-resolution icon set from the real logo: favicon.ico, favicon-32.png, icon-192.png, icon-512.png and apple-touch-icon.png, with manifest references wired in index.html. |
| Header branding | The navigation bar had no project mark. | Navbar.jsx renders logo-mark-128.png on desktop and logo-mark-64.png in the mobile view, in both light and dark themes. |
| Result actions | Action labels wrapped to two lines at narrow widths, misaligning the row. | whitespace-nowrap and non-shrinking icons applied to the report, download and delete controls. |
| Doctor search layout | The search field occupied only one column beside the doctor directory. | The field now spans the full directory width. |
| Login error handling | A mistyped password left a persistent blocking alert, and a genuine 401 from the interceptor could not be distinguished from a credential rejection. | The alert auto-dismisses after three seconds; credential rejections return HTTP 400 while 401 is reserved for expired tokens. |
| Plaintext database credential in source | app/core/config.py carried a fallback connection string containing a real password, so the secret was distributed with the source and would ship with any clone. | The fallback was removed. An explicit sentinel is the only default, and the service refuses to start if DATABASE_URL is still set to it. The value now lives solely in an untracked .env file, which git ignores. |
| Clinical dashboard open to administrators | The doctor dashboard admitted admin accounts, but every action the screen offers is doctor-gated, so an admin saw a cockpit of controls that each returned 403. | The handler now returns 403 for any non-doctor, matching its own action set. Administrators use the /admin surface. Regression tests assert admin 403, doctor 200 and patient 403. |
| Duplicated route handler | POST /api/doctor/link-prediction-to-patient was declared twice with an identical body, so the second copy was unreachable and could silently diverge. | The redundant declaration was deleted and the surviving handler kept, including its prediction-ownership check. See Section 9.11. |
| Patient identity leak on the public queue | The unauthenticated waiting-room route returned the patient's full name and the consultation fee to anyone who asked, which disclosed protected health information on a public surface. | The response now carries the token number, the status and an initials-only patient label, and the fee is withheld. Three regression tests assert that neither the full name nor a fee field can appear. |
| No administrative oversight of clinical records | An administrator could see aggregate counts but had no way to inspect or correct an individual prediction or booking. | Four admin-only endpoints were added to list and delete predictions and appointments, with pagination and joined patient names. Deleting a prediction also removes its radiograph and Grad-CAM overlay. Doctors and admins are separated, verified with a 403 probe. |

| Area | Symptom | Resolution |
|---|---|---|
| User deletion leaked stored media | Deleting a user cascaded the database rows but left every radiograph and overlay the user had ever uploaded on disk. | The media paths are collected before the bulk delete and the files are removed afterwards through the same helper the prediction endpoint uses. |
| Admin edits could create duplicate identities | CNIC and phone were each unique within their own column only, so one user could be given another user's CNIC while keeping a different phone. | Both fields are now validated as a pair, with a digits-only format check, and a collision returns 409 with no partial write. |
| Contact settings accepted arbitrary values | The contact-settings endpoint stored whatever it was given, so a malformed phone number or an invalid address could be published on the public contact page. | Emails and phone and WhatsApp numbers are validated, the latter as 10 to 15 digits with spaces and punctuation stripped. Every field is validated before anything is written, so a rejected update leaves the previous values intact. |
| Raw Axios calls bypassed the interceptor | TodayTokens.jsx and DoctorDashboard.jsx each built their own Authorization header, duplicating token handling and bypassing the shared 401 behaviour. | Both pages now use the shared client in services/api.js, so token attachment, base URL and expired-session handling live in one place. The dashboard's WebSocket keeps its own token because the browser WebSocket API cannot set headers. |

### *14.2 Policy and content changes*

- Public registration was restricted to doctors holding a valid single-use access code. The backend rejects any non-doctor role at the registration endpoint, and patients are now provisioned by a verified doctor or through the guest walk-in flow.
- Terms and Privacy were expanded from three to sixteen sections each, with consistent Lucide iconography. These pages contain institutional policy commitments that require faculty and institution review before publication.
- The database was reset to a clean demonstration state with a pre-wipe backup taken. It now holds three users, one doctor-patient link, three synthetic predictions and five site settings, and no appointments, guest requests or contact messages.

### *14.3 Defects identified and deliberately not changed in this pass*

| Observation | Location | Status |
|---|---|---|
| An identical route handler was declared twice, leaving unreachable dead code that could silently diverge from its twin. | routes.py | RESOLVED. The redundant declaration was deleted; the surviving handler keeps its prediction-ownership check. See Section 9.11. |
| Eight orphaned _gradcam.jpg files remain in the radiology upload directory from before the deletion fix. No user record references them, so no live request can reach them. | backend/uploads/radiology | Left in place. A storage audit was not requested. |
| No route-level React error boundary exists, so a render-time exception blanks the whole view instead of showing a recoverable message. | frontend/src | Recommended in Section 15. |

# 15. Appendix B: Known Limitations & Recommended Next Steps

## 15.1 Current limitations

| # | Limitation | Impact |
|---|---|---|
| 1 | No frontend or end-to-end browser tests. | Interface regressions are caught only by manual verification. |
| 2 | The six-stage CV gate is untested. | Rejection thresholds are unverified against real radiographs and adversarial inputs. This is the highest-value testing gap. |
| 3 | SQLite is used for tests while production runs PostgreSQL. | Concurrency and constraint behaviour specific to PostgreSQL is not exercised. |
| 4 | Admins have no write oversight of clinical records. | Mitigated: admins can now list and delete predictions and appointments, including removal of the stored radiograph and Grad-CAM overlay. They still cannot alter a diagnosis or a fee. |
| 5 | Access-code redemption is guarded by a process-local set plus a unique index. | In a multi-instance deployment, simultaneous redemption of one code surfaces a duplicate-key error rather than a clean rejection. |
| 6 | No application-layer rate limiting. | Login, contact and walk-in endpoints rely on the reverse proxy for throttling. |
| 7 | No content-security-policy header. | Existing security headers do not constrain script sources. |
| 8 | The model checkpoint's provenance is not versioned or checksummed. | Published accuracy figures cannot be reproduced from the tree alone. |
| 9 | No inference latency or throughput benchmark is recorded. | Capacity planning for a clinic deployment is not evidence-based. |
| 10 | No HTTPS enforcement in the application layer. | Correct in the Compose stack where Nginx terminates TLS, but a bare Uvicorn deployment would serve tokens in clear text. |
| 11 | Guest status is polled by identifier, which is enumerable. | A caller can probe whether an arbitrary phone or CNIC has a pending request. |
| 12 | Terms and Privacy commitments are not yet reviewed by the institution. | Encryption, storage jurisdiction and human-review claims must not be published as verified controls until confirmed. |

## 15.2 Recommended next steps, in priority order

1. Add image fixtures for the six-stage CV gate: a representative set of genuine chest radiographs plus adversarial cases including selfies, photographs of people, landscape scenes, blank uploads and low-light captures. This converts the safety gate from a heuristic into a regression-tested component.
2. Add a test that asserts no path and method pair is registered twice, so the class of defect cannot return. The duplicate handler itself has already been removed.
3. Introduce a route-level error boundary so a render exception is contained and recoverable.
4. Propagate access-code redemption through the database by claiming the code with a conditional update, which makes multi-instance deployment safe.
5. Add rate limiting to the login, contact and walk-in endpoints, and set a content-security-policy header.
6. Adopt Alembic migrations as the single source of truth for schema changes rather than relying on implicit table creation.
7. Introduce a frontend test layer beginning with the clinically critical paths: upload, result rendering and token booking.
8. Record model provenance, training configuration and evaluation metrics alongside a checkpoint checksum.
9. Conduct a performance assessment of inference latency and WebSocket fan-out under a realistic clinic workload.
10. Sweep the orphaned overlay files identified in Section 14.3 and add a startup reconciliation step that reports unreachable media files.

**Closing statement.** The platform is functionally complete across authentication, explainable inference, OPD queue management and administration, and it is guarded by a 61-test regression suite. The most valuable remaining work is not feature development but hardening the two places where the system makes a clinical claim: the image sanity gate and the model-connection invariant.

# 16. Appendix C: Viva Voce Question Bank

This bank is drawn entirely from the verified behaviour of this codebase. Answers are written in the form they could reasonably be spoken, so they can be rehearsed rather than memorised. Where the honest answer is a weakness, the weakness is stated plainly, because an examiner probes a claimed strength far harder than a conceded limitation.

**How to use this bank.** Nineteen categories, 160 questions. A and B cover the scope and the justification, and B is the one to rehearse hardest, because the first question any examiner asks about a clinical AI project is why the model should exist when a doctor is already looking at the image; C, D and E cover the model, the explanation and the image gate; F to I the engineering, data, security and interface; J and K testing and operations; L and M the subjective questions about what went wrong and what you would change; N and O the two deep-dive sets, fifteen backend and nineteen frontend questions; P the thirteen questions on how the two halves of the system talk to each other; and Q, R and S the thirty questions drawn from the submitted write-up, covering the experimental evidence, the literature review and the dataset and training protocol. Rehearse B, L, M, P and Q first. They are where a candidate who has only memorised the feature list is exposed, and Q is the most dangerous category in the bank, because an examiner holding your write-up can add the numbers up, and in this project they do not all agree. Every answer in Q and S is written to be defensible under exactly that scrutiny, which means several of them concede a fault in the document. That is deliberate: a prepared concession scores far better than an improvised defence of a number that does not survive arithmetic.

## A. Project scope, motivation and problem statement

### *Q. Why did you choose this project?*

Tuberculosis and pneumonia remain leading infectious causes of death worldwide, and in Pakistan and much of the developing world the bottleneck is not the absence of radiology expertise but its availability. A chest radiograph is the cheapest, fastest and most widely available investigation for both diseases, yet a single qualified radiologist can only read a limited number of films per day, and rural clinics often have none at all. I wanted to build something that does not replace the radiologist but removes the queue in front of them: a screening layer that triages the ordinary cases so the clinician spends attention on the abnormal ones. That framing is the whole project. It is triage software, not a diagnosis engine, and every screen in the interface says so.

### *Q. What is the actual problem your system solves?*

Three problems, in order of importance. First, throughput: a film that needs a human opinion waits in a physical or telephone queue, and in a typical OPD the waiting is far longer than the reading. Second, consistency: junior readers vary in what they call normal. Third, access: a patient in a remote location cannot get a chest film interpreted at all. The system addresses the first with an automated pre-screen, the second by giving the reviewing clinician a consistent second signal with a localised heatmap, and the third by making the interpretation capability deliverable over a network connection rather than requiring the radiologist to be in the room.

### *Q. Who are the users of your system?*

Four, with genuinely different needs. The patient, who may be a registered account holder or an anonymous walk-in, uploads a film and needs a clear, non-alarming result. The doctor, who is the primary daily user, runs the clinic from a cockpit: call the next patient, approve walk-ins, collect fees, review results. The admin, who manages users, global settings and the contact inbox. And the waiting room, which is a display rather than a user, showing live queue state. That last one is why the queue is a first-class part of the system rather than an afterthought.

### *Q. Is this a replacement for a doctor?*

No, and I want to be precise about why, because it is the most important limitation in the project. The model outputs a class and a probability from a single frontal chest radiograph. It does not see lateral views, it has no clinical history, no lab data, and no comparison with prior films. It cannot distinguish active from inactive tuberculosis, and it cannot exclude malignancy. A screening result is a triage signal, so the interface presents it as advisory, the heatmap shows the model where it looked, and the reviewing doctor makes the decision. Anything else would be an unsafe claim.

### *Q. What is the single biggest limitation of your system?*

The image validation gate is untested. It has six heuristic stages, including a face cascade and a structural edge check, and none of them has a unit test with real positive and negative fixtures. It is a heuristic, so it can be evaded, and I cannot demonstrate its false-rejection rate. I rank this above model accuracy deliberately, because accuracy is measurable and I have measured it, whereas the gate is an unmeasured component sitting in front of the model.

### *Q. What would you build next, if you had another month?*

In priority order: unit tests for the six-stage gate with a real fixture set, because it is the least-verified component in the critical path; a route-level error boundary on the frontend, since one render exception currently blanks the entire view; and a second radiograph view, because a lateral view would meaningfully change what the model can exclude. The duplication defect I found in the route table is a ten-minute fix, but I would also add a test asserting no duplicate route registration.

## B. Defending the premise: why train a model at all?

### *Q. If a doctor checks the X-ray anyway, why do we train the model at all?*

That is the right question to ask, and the honest answer is that the doctor does not check every film. In the setting this project targets, a chest radiograph is ordered, it goes into a queue, and a large proportion is never individually reported, or is reported in a batch at the end of a clinic session. So the model is not competing with the doctor for the films the doctor definitely reads. It is deciding what the doctor looks at first. That reframes the whole thing: this is a worklist prioritisation problem, not a diagnosis problem. The model reads every film, cheaply and immediately, and the radiologist's attention goes to the ones it flags. If the model never changed the order in which the doctor worked, it would indeed be pointless, so that is precisely the thing I would want to measure.

### *Q. Then what does the model actually contribute, if the doctor makes the final decision?*

Three concrete things. Throughput: it removes the queue in front of the radiologist, so the binding constraint, which is expert hours, is used better. Consistency: a junior reader and a consultant differ on what counts as normal, whereas the model applies identical criteria to every film. And access: at a clinic with no radiologist on site, a network connection now delivers a second reading, which previously simply did not exist. Note what the model is not doing: it is not deciding the outcome, and it is not removing the doctor from the loop. The decision is still human, and the model's job is to make that decision cheaper and better prioritised.

### *Q. Why use deep learning rather than classical machine learning on hand-crafted features?*

Because the features I would hand-craft are the ones I cannot specify. A radiologist's description of consolidation is not a measurable threshold, it is a pattern recognised across texture, shape and location simultaneously. Encoding that as explicit features means spending months designing detectors and still missing the combinations. A convolutional network learns hierarchical features from the data instead, and the DenseNet dense connectivity is specifically good at reusing the fine-grained detail that faint findings depend on. The honest counter-argument is that a well-tuned classical model on good features could be competitive with far less data, and with this project I have more data than I can rigorously validate on.

### *Q. The system already rejects bad images with the validation gate. Is not that the AI doing the work?*

No, and the distinction matters. The gate is deterministic image processing: it measures dimensions, sharpness, dynamic range, chrominance, faces and edge structure, and it applies fixed thresholds. It cannot tell you what the disease is; all it can do is say whether the image is suitable for inference. It is a filter, and I described it as a heuristic that can be bypassed. The model is what interprets the image. They are complementary and they fail differently, which is exactly why having both is better than having either: the gate catches the photograph, the model catches the consolidation.

### *Q. If the result is only advisory, what stops clinicians simply ignoring it?*

Nothing enforces it, and I think that is correct. I have not built an override because a system where the machine can be overruled still has to be right often enough to be worth reading, and building in an obligation to agree would be worse than useless because it would produce rubber stamping. What the design does try to do is make ignoring it a deliberate act rather than the default. The heatmap gives a clinician something concrete to disagree with, the full probability distribution shows when the model is genuinely uncertain rather than confidently wrong, and the wording is advisory throughout. Adoption in a real clinic would be a change-management problem, not a technical one.

### *Q. Why is this a screening tool and not a diagnostic one? What is the difference?*

Screening answers a triage question: does this person need to be seen urgently? Diagnosis answers aetiologic and management questions. The distinction is concrete in this system. The model sees a single frontal projection, so it cannot assess the retrocardiac region well, cannot use a lateral view, has no clinical history, no laboratory data, and no prior film for comparison. It cannot distinguish active from inactive tuberculosis, cannot grade severity, and cannot exclude malignancy or anything outside its three classes. Anything it says is a reason to look closer. That is the line I hold, and every screen in the interface is worded to keep it.

### *Q. Is the model just predicting 'normal' most of the time and looking accurate?*

That is a real risk with imbalanced medical data, and it is the first thing to check on any chest X-ray classifier, so I would expect the question. In a general screening population the normal class is the majority, so a model that leaned on the majority class could look acceptable on accuracy while being clinically useless, because the cases that matter are the minority. That is precisely why the system does not report a bare accuracy figure: the full three-class distribution is stored and displayed for every prediction, and a per-class breakdown with a confusion matrix is what should be judged. I would consider a model that achieved good accuracy by favouring the majority class to have failed, regardless of the headline number.

### *Q. Why build an entire clinic management system when the core of it is a classifier?*

Because the classifier alone does not change any clinical outcome. A prediction that arrives with no patient attached, no review workflow, no audit trail and no way to escalate is a research demo. The value is created in the surrounding system: the authenticated access that makes the record legitimate, the review step where a clinician signs off, the OPD queue that gets the flagged patient seen faster, and the immutable history that lets a later clinician compare. The model is the part that is interesting to write about and the smallest part of what makes the system work.

### *Q. If Grad-CAM only shows where the model looked, why not just show the doctor a higher-resolution image?*

Both, and they answer different questions. The higher-resolution image is the evidence; the heatmap is the model's reasoning. When they agree, the clinician has a quick confirmation and can move on, which is where the throughput saving comes from. When they disagree, that is the most informative case on the screen, because it flags a film where the model's attention did not match the anatomy, which usually means a technical artefact, an unusual projection, or a genuine error. A resolution increase cannot tell you that. I would not present the heatmap as a finding, though, and it must never replace looking at the film.

### *Q. What would convince you to abandon the neural network and go back to rules?*

If the validation set showed the network adding nothing over the gate plus a radiologist, meaning its flagged set was no more enriched for abnormality than an arbitrary queue order, then the engineering and the maintenance cost would not be justified. Concretely I would want to know whether films the model flags are genuinely more often abnormal than films it clears. If that enrichment were negligible, a simpler triage rule would be preferable, because it would be explainable to a clinician without a training set. The network earns its place by improving that enrichment, and that is the single measurement I would most want to publish.

### *Q. How do you know the model actually helps, rather than just adding a step?*

Honestly, the strongest evidence would be a study, and I have not run one. What I can argue is mechanistically: the clinician's time is the scarce resource, so the question is whether predicting abnormality lets them clear normal films faster without missing a case that matters. That needs a comparison of reading time and missed findings with and without triage, ideally a prospective one. Within this project I have verified the model is stable, explainable and never fabricates output, but I have not demonstrated a clinical benefit, and I would not claim one without that measurement.

## C. Deep learning model

### *Q. Why DenseNet-121, and not ResNet or a plain CNN?*

DenseNet's defining feature is that every convolutional block receives the concatenated feature maps of all preceding blocks, so features are reused rather than re-learned, and the gradient path is short. That matters specifically for radiographs. The findings I care about, consolidation and apical cavitation, are often faint, low-contrast and spatially distributed, so the ability to reuse fine-grained features instead of compressing them away is valuable. ResNet solves degradation with residual addition, which also works, but DenseNet's concatenation tends to preserve more detail per parameter. I retargeted the final fully connected layer from 1000 ImageNet classes to 3 with nn.Linear(num_ftrs, 3).

### *Q. Walk me through what happens to an image the moment it is uploaded.*

The extension is checked against jpg, jpeg and png, the bytes are read, and a 10 MB ceiling is enforced. The file is written under a fresh UUID so no attacker-controlled path survives. Then the six-stage OpenCV gate runs; if it fails, the response is HTTP 400 naming the specific stage that rejected the image, so the user learns whether to retake the film. If it passes, the model connection invariant is checked, the image is resized to 224x224 and normalised with ImageNet statistics, and a forward pass produces three logits. Softmax gives the distribution, arg-max selects the class, and Grad-CAM is computed for the winning class. Everything is persisted, and the images are retrievable only through the authenticated media route.

### *Q. Why 224x224, and what happens when you resize a chest X-ray to that?*

224x224 is the input resolution DenseNet-121 was trained at, and preprocessing must match training exactly or the accuracy measured offline will not transfer. The cost is real and I will not pretend otherwise: a chest radiograph is roughly square, and normalising a tall film into a square compresses the lung fields vertically, so fine detail is lost. It is an accepted trade-off of using an ImageNet-pretrained backbone. I deliberately do not use a RandomResizedCrop at inference, because a stochastic transform would make the reported confidence non-reproducible for the same file, which is unacceptable in a diagnostic context.

### *Q. What are your three classes, and why exactly three?*

Normal, Pneumonia and TB, at class indices 0, 1 and 2. I merged all normal studies into a single normal class rather than keeping separate normal variants, because the triage decision is binary in practice: does this film need urgent human attention or not. Three classes keeps the decision boundary sharp and avoids splitting a clinically equivalent group across labels, which would waste capacity on a distinction no clinician acts on. The model reports the full probability vector as well, so no information is lost.

### *Q. How does the model produce a confidence value, and is it a real probability?*

The forward pass gives three logits, I apply softmax to get a distribution, then take the arg-max entry as the confidence. Two honest caveats. Softmax confidence is better calibrated than raw logits but is still not a calibrated estimate of real-world accuracy, because a model trained on one dataset is usually overconfident on out-of-distribution input such as a film from a different machine. And a high softmax score only means the model is internally confident, not that it is correct. That is precisely why I show the whole distribution and the heatmap rather than a single percentage, and why the label reads as a screening indication.

### *Q. Why does the system refuse to predict when the model fails to load?*

Because the alternative is a system that lies. If weights are missing, a plausible-looking random label would be indistinguishable from a real result at the interface, and in a medical context that is the worst possible failure mode: silent, plausible and wrong. So predict_image raises when the model is disconnected, the route returns HTTP 503, and the frontend disables the analysis control until the public model-status endpoint reports a connected model. The trade-off is explicit: a disconnected model blocks screening entirely rather than degrading quietly. For a triage tool I would rather stop than guess.

### *Q. How large is the model, and where does it run?*

The checkpoint is 28,433,177 bytes, about 28 MB, and it loads onto the CPU with map_location, so the service runs on hardware without a CUDA runtime, which matters for a clinic deployment. The connection state is tracked in a module-level status dictionary that the model-status endpoint reads, and the weights are loaded once at startup rather than per request.

### *Q. What is your accuracy, and how did you measure it?*

I should answer this carefully. The reported figures come from evaluating the checkpoint on a held-out split during the offline training work, not from this repository, because the delivered tree contains the inference path only and no evaluation script. That is a genuine weakness in the deliverable: the numbers are not reproducible from the tree, the split is not recorded here, and there is no per-class breakdown or confusion matrix in the repository. I have flagged it in the limitations as the first thing to fix, ahead of any new model work.

## D. Grad-CAM and explainability

### *Q. Why add Grad-CAM when the classifier already gives a label?*

Because a bare label is not clinically actionable. If the system says pneumonia with 0.94 confidence and nothing else, the doctor's only options are to trust it or ignore it. Grad-CAM answers a different question: where did the model actually look? A heatmap concentrated in the right lower zone gives a clinician a reason to engage with the result, and a heatmap scattered across the whole frame is a visible warning that the model may be responding to something spurious. It converts an assertion into a checkable claim.

### *Q. Explain how Grad-CAM works in this project.*

I retain the activation map from the final convolutional block, which has shape N by C by H by W. I backpropagate the target-class score to that same block, average the gradients over the spatial dimensions to get one importance weight per channel, and take a weighted sum of the activations followed by ReLU. That yields a coarse single-channel heatmap. I normalise it to the range zero to one, upscale it to the original resolution, apply an OpenCV JET colour map, and composite it over the radiograph. The key detail is that the target class is the predicted class, so the map explains the answer that was actually given.

### *Q. How should a clinician read the heatmap for TB versus pneumonia?*

Tuberculosis classically favours the apical and upper-zone fields where cavitation develops, so a TB attribution should concentrate superiorly. Pneumonia presents as patchy, often asymmetric consolidation, typically lobar, so attention should be regional and one-sided rather than diffuse. The important caveat is that this is a model-attribution aid, not an independent second opinion. It shows where the network's evidence came from, and a heatmap can look anatomically plausible while the underlying decision is wrong, so it must never be presented as confirming a finding.

### *Q. How are heatmaps stored, and what bug did you hit there?*

Each overlay is written beside its original as the original filename stem plus the suffix _gradcam.jpg, and it is never referenced by a database column. That design is compact but it created a real bug: the media route originally resolved the file by matching image_path against the requested filename, and because the stored path never contains the overlay filename, every single heatmap request returned 404. The explainability feature was completely unreachable and looked like it worked, because the page rendered and only the image failed. The fix was to strip the _gradcam suffix, resolve the owning prediction, re-verify authorisation, then serve the requested variant. I wrote the tests first, watched them fail against the old logic, and only then applied the fix, because a test written after the fix would not have proved anything.

## E. Computer vision pre-screening

### *Q. How many validation stages are there, and what does each reject?*

Six, in image_validator.py. One checks dimensions and aspect ratio, rejecting thumbnails, icons and extreme panoramas. Two checks focus using the variance of the Laplacian, catching out-of-focus and motion-blurred captures. Three checks dynamic range and radiographic contrast, rejecting flat grey uploads and blank film. Four checks chrominance and skin, using saturation in HSV plus a skin-chrominance mask in YCrCb and HSV, which rejects colour photographs and heavily tinted scans. Five runs a Viola-Jones face cascade, refined by eye detection inside the face region, to reject selfies and portraits. Six verifies structural edge density across the thoracic field to catch natural scenes that survived the first five. The function returns a reason string, so the user is told which stage objected rather than just seeing a failure.

### *Q. Why use a Laplacian variance to detect blur?*

The Laplacian kernel is a second-derivative operator, so it responds strongly to intensity changes and produces near-zero output across smooth regions. A sharp radiograph has many edges, so the variance of the Laplacian response is high; a blurred image suppresses fine edges, so that variance collapses. It is cheap, parameter-light and well suited to a per-request budget, which is why it beats a heavier sharpness metric here. I apply it to a greyscale copy, and it is a threshold, so it is a heuristic that can be evaded rather than a guarantee.

### *Q. Why go to the trouble of face detection on a medical upload?*

Because the most likely bad input is not an attack, it is a well-meaning user photographing something on a phone screen. In practice the common failure is a selfie, a photograph of a person, or a picture of a printed report. A CNN will happily emit a confident label for any of these, because the input is in-distribution enough to produce activations, and the resulting output looks identical to a real result. Detecting a face and, crucially, eyes within that face catches the photograph case specifically, which a generic colour check misses because a greyscale photograph of a person is monochromatic.

### *Q. What is the weakest of the six stages?*

Stage six, structural edge verification. Stages one to five are grounded in measurable quantities, but stage six checks for rib, spine and clavicle edge signatures, and I have no validated threshold for it. A chest film with unusual exposure, a portable paediatric film, or a rotated image could plausibly fall outside it. That is the stage I would instrument first if I had more time, by logging which stage rejected which image and reviewing the rejections, which is how the threshold would be set from data instead of judgement.

### *Q. Can an attacker just bypass the gate?*

Yes, and the documentation says so explicitly. The gate is a heuristic filter whose purpose is to reduce wasted inference and catch obvious user error; it is not a security control. The hard guarantee in this system is not the gate but the model-connection invariant, which prevents a fabricated result regardless of what the image contains. If the gate is bypassed, the worst outcome is a low-quality prediction that a reviewing clinician should catch, not a system compromise.

## F. Backend, API and architecture

### *Q. What is the backend built with, and why FastAPI specifically?*

FastAPI on Uvicorn. Three reasons were decisive. It is async, so a route that awaits file reads does not block the event loop while a clinic has many users on the waiting-room display. It is typed, so Pydantic validates request bodies and generates OpenAPI documentation automatically, which means the API contract cannot drift from the implementation. And its dependency injection gives a single place to resolve the bearer token and the database session, so authorisation is declared per route rather than scattered through handler bodies.

### *Q. What does the decoupled architecture actually buy you?*

The frontend and backend share no code and no memory. The React application is a static bundle served independently, and the API is stateless, so the two can be deployed, scaled and restarted separately, and a backend restart does not log anybody out because the session lives in a signed token rather than in server memory. The one deliberate exception is the doctor access-code burn set, which is process-local, and I treat that as a documented weak point rather than pretending the system is stateless.

### *Q. How many endpoints are there, and how did you count?*

Fifty-one route handlers: forty in api/routes.py plus one WebSocket, and ten in the separate admin router. I counted decorators in the source rather than trusting the previous documentation, which claimed thirty. Doing that count also surfaced a defect: one path, POST /doctor/link-prediction-to-patient, is registered twice, at line 175 and again at line 1009, with identical function names and bodies. FastAPI matches in registration order, so the second is unreachable dead code that can silently diverge from its twin. I documented it rather than fixing it silently.

### *Q. How does the frontend know the API is on a different port?*

One environment variable, VITE_API_URL, and the client derives everything else from it. The configured value has any trailing /api stripped to get the server origin, and the API base is re-appended. That yields two constants, one for XHR calls and one for media files, so the origin is never configured twice and cannot drift between them.

### *Q. Why is there a WebSocket when you could just poll?*

Because polling is wasteful and visibly laggy for a waiting room. A queue board that updates every few seconds is both behind the truth and generating constant avoidable load, and with a dozen connected displays that multiplies. The WebSocket pushes state changes as they happen, so latency drops to roughly ten milliseconds. One important design decision: the payload is treated as a signal, not as canonical data, so clients re-fetch authoritative state on receipt rather than trusting the broadcast to be complete. That costs a small request per event and removes a whole class of desynchronisation bug.

### *Q. The WebSocket could not access the database. How did you fix that?*

Route handlers receive the database session through FastAPI dependency injection, which runs per request. A WebSocket is a long-lived connection with no request scope, so there is no session to inject and the handler had nothing to work with. The fix was to publish the session factory on application state at startup and let the WebSocket open its own short-lived sessions, rather than sharing one session across all connections, which would have been unsafe under concurrency.

### *Q. How did you handle file uploads?*

As multipart form data through UploadFile, which FastAPI streams rather than buffering into memory, and the frontend uses FormData without ever setting the Content-Type header. That last point was the source of a real bug: five call sites set Content-Type manually, which strips the multipart boundary, and the server then rejects the payload with 400 Missing boundary in multipart. The misleading part is that this surfaces in the browser console as a CORS error, so it sends you chasing the wrong problem entirely. The rule is that the runtime must supply the header whenever the body is FormData.

## G. Database design

### *Q. How many tables, and why is one users table enough for three roles?*

Seven tables, with a single users table discriminated by a role column rather than separate doctors, patients and admins tables. The justification is that all three share identity attributes, phone and CNIC, that must be unique across the whole system; a login form that accepts either identifier has to resolve against one namespace. Splitting into three tables would mean a union or a view to search for a login, and the uniqueness constraint would have to be duplicated across all three. The doctor-only columns are nullable, so patients simply leave them null.

### *Q. What is the doctor_patients table for?*

It is an explicit junction table modelling a genuine many-to-many relationship, because a patient can be under the care of several clinicians and a doctor can follow several patients. It is separate from appointments on purpose, because care continuity outlives any single visit, and deriving the relationship from appointment history would make it impossible to record that a doctor is managing a patient who has not been seen yet.

### *Q. Why does an appointment store fee_charged when the doctor already has consultation_fee?*

Because the fee is snapshotted at booking time. If the doctor raises the price, patients who already booked must still be charged what they were quoted, and a system that read the current fee would silently change existing obligations. Copying the value onto the appointment makes the financial record self-contained and auditable, which matters more in billing than avoiding one duplicated column.

### *Q. What do token_mode fixed and dynamic actually do?*

Fixed mode allocates upward from one, so display order matches arrival order, which is what a patient expects at a normal clinic. Dynamic mode allocates downward from a high base, so the lowest number is always the current serving patient, which suits a department where the next patient is often whoever is physically closest. It is a per-doctor preference because both are legitimate and clinics genuinely differ.

### *Q. How do you stop two patients getting the same token number?*

Allocation re-reads the highest issued token for that doctor and date inside the transaction and increments from it, so the number is derived from committed state rather than from a client counter. I would be honest that this is not as strong as a database sequence per doctor-day, and under genuine concurrent load on PostgreSQL two transactions can still read the same maximum. The belt-and-braces fix is a unique constraint on doctor, booking date and token number, which I would add next.

### *Q. How does deleting a prediction clean up?*

The delete endpoint removes the database row and then deletes both the radiograph and the paired _gradcam overlay from disk. Before the fix it removed only the row, which orphaned both files and left unreferenced clinical images accumulating. I also corrected the button wording, which had said Archive while performing a permanent delete, because a destructive irreversible action must not be described in reversible terms.

### *Q. What is wrong with how the schema is managed?*

Honestly, there is no migration system in the delivered project. Alembic is in the requirements, but there is no alembic.ini and no migrations directory, and the schema is created by create_all at application startup. That works for a fresh database and is fine for a demonstration, but it cannot express an alteration to an existing table, and create_all will not drop or reshape a column. For anything beyond a prototype I would initialise Alembic, generate a baseline revision, and make migrations the only way the schema changes.

## H. Security

### *Q. How does authentication work?*

Passwords are hashed with bcrypt, which generates a random salt per password, so two users with the same password have different hashes and precomputed rainbow tables are useless. Login returns an HS256 JWT carrying the subject, role, primary key and expiry. Every protected route resolves the token through one dependency that rejects a bad signature or an expired token with 401. The role is read from the signed token, never from anything the client sends separately.

### *Q. What stops a doctor from changing their own role in the token?*

They cannot, because the role claim is inside the signed payload and the signature covers the whole payload; altering the role invalidates the signature and the request is rejected with 401. This is exactly why role-based authorisation must read the role from the verified token on the server and never trust a role supplied in a request body, a query parameter or a client header.

### *Q. Why restrict public registration to doctors?*

Because an open patient sign-up would let anyone create an identity and upload arbitrary images, and a screening system accumulates medical records. Instead a new doctor must present a one-time access code, and a patient account can only be created by an already verified doctor or through the guest walk-in flow. The code is recorded in a uniquely indexed column, so a code cannot be redeemed twice even if the process restarts and loses the in-memory burn set.

### *Q. What is wrong with the access-code implementation in a multi-instance deployment?*

The redemption guard is a process-local set backed by a unique index. The unique index is the correctness guarantee and it holds, but redemption is not propagated between instances, so if two application instances handle the same code simultaneously one of them will hit a duplicate-key error rather than a clean rejection with a helpful message. The correct fix is to claim the code with a conditional update, where the update succeeds only if the code is still unclaimed, making the database the single arbiter.

### *Q. How are medical images protected?*

This was the most significant security change in the project. Diagnostic radiographs are not mounted as static files, so there is no public URL to guess or share. They are streamed only through a dedicated media route that requires a token and re-verifies record ownership on every single request. The previous documentation described static upload directories being served with a wildcard CORS origin, which would have exposed every patient's radiograph to anyone who could reach the server.

### *Q. Images need a token, but an img tag cannot send a header. How does that work?*

The route accepts the bearer token as a query parameter and re-verifies it, because an HTML image element cannot attach an Authorization header. The trade-off is that query strings appear in access logs and browser history, so a production system should prefer a short-lived signed URL or a cookie, or fetch the image through fetch with a header and render it from a blob URL. What I would not accept is a static mount with a permissive origin.

### *Q. What other security controls are in place?*

Uploads are restricted to jpg, jpeg and png extensions, capped at 10 MB, and stored under a freshly generated UUID filename so no attacker-supplied path component survives. The media route enforces basenames and verifies the resolved path stays inside the uploads directory, which blocks traversal. Security response headers are applied by middleware: nosniff, X-Frame-Options SAMEORIGIN, X-XSS-Protection and HSTS. CORS uses an explicit allow-list rather than a wildcard, because credentialed requests cannot be combined with a wildcard origin.

### *Q. What security gaps remain?*

Four that I am aware of. There is no content-security-policy header, so script sources are not constrained. There is no application-layer rate limiting, so login, the contact form and the walk-in request are throttled only by whatever the reverse proxy does. File type validation inspects the extension rather than the magic bytes, so a renamed file is accepted and then fails at decode. And guest registration status is polled by phone or CNIC, which is enumerable, so a caller can discover whether an arbitrary identifier has a pending request. None of these give data disclosure, but all of them are real.

### *Q. Is storing a JWT in localStorage a problem?*

It is a trade-off and I will defend it rather than claim it is ideal. localStorage is readable by any script on the origin, so it is exposed to XSS, whereas an httpOnly cookie is not. The mitigation is that React escapes rendered text by default, so the realistic XSS surface is small, and I mitigate further with the security headers. The stronger design is an httpOnly, Secure, SameSite cookie with CSRF protection, which I would move to if I were extending the system, along with short-lived access tokens and refresh tokens, which is why the token lifetime is kept short.

## I. Frontend

### *Q. Why React, and what does the state model look like?*

React 18 with function components and hooks, built by Vite. I chose React for the component model and the ecosystem, and Vite because its dev server start time and hot reload are dramatically faster than a webpack-based setup, which matters when iterating on an interface this size. State is deliberately local: server data is fetched per page and a small AuthContext carries identity and role. I avoided a global state library because the app has few genuinely shared pieces of state, and most screens are self-contained.

### *Q. How do you protect routes in the frontend, and why is that not enough?*

ProtectedRoute wrappers check the role from the auth context and redirect to login, which prevents a user navigating to a portal they should not see, and gives a better experience than letting them load a page of failed requests. But it is purely a user-experience measure. Everything is client-side, so an attacker can simply call the endpoint directly, which is why every authorisation decision is re-made on the server from the signed token. Treating the frontend guard as security is the classic mistake this project is structured to avoid.

### *Q. How does the theme switch work?*

A ThemeContext holds the current mode and applies it at the document root, so both the light and dark clinical themes are available and the choice persists. The same applies to the branding: the navigation renders the real project mark, using a 128-pixel asset on desktop and a 64-pixel asset in the mobile view, and I replaced the default Vite favicon that was previously showing in the browser tab.

### *Q. What is Framer Motion used for, and is animation just decoration here?*

Mostly transition and hierarchy: card entrances, the confidence meter filling, and modal choreography, so that a state change is noticed. The exception is the waiting-room queue, where row order carries meaning and an animated reorder makes a change in priority legible. Beyond that I kept animation out of the clinical readouts, because in a result view a moving element competes with the finding for attention.

### *Q. How do you generate the downloadable report?*

Client-side with html2pdf.js, which renders the result view into an A4 document. It is appropriate for a summary the patient takes away, and it means no report data passes to a third-party service. The limitation is that it rasterises through the browser, so the output is image-like rather than selectable text, and pagination control is weak. A server-rendered PDF would be the choice for an official clinical document that needs to be archived and searchable.

### *Q. Why is there no error boundary?*

That is a gap I found during review and did not fix in this pass. Without a route-level error boundary, a render-time exception unmounts the whole view and the user sees a blank page with no way to recover except a manual refresh, which is indistinguishable from the app being broken. A small boundary component that catches the error, shows a recoverable message and offers a retry is a contained fix and is on my list.

## J. Testing and quality assurance

### *Q. How do you test a medical application?*

The same as any application, with more attention to the negative and authorisation paths. In a diagnostic system the dangerous outcomes are not wrong predictions, which a clinician will catch, but a fabricated result, a leaked radiograph, or a patient seeing another patient's data. So the suite concentrates on authentication and role enforcement, on the media route's ownership checks including denial of cross-patient access, and on the queue state machine. The suite is thirty-four tests across three modules and all of them pass.

### *Q. What do the tests actually cover?*

Authentication and security cover registration role enforcement, access-code validation and replay rejection, login by both phone and CNIC, wrong-password handling, JWT claim integrity, expiry, and 401 behaviour for missing or malformed tokens. Media covers original serving, Grad-CAM resolution for both PNG and JPEG overlays, denial of cross-patient access, traversal rejection, missing files, query-token authentication, and that deletion removes both files. Queue covers allocation in both token modes, capacity and queue-window enforcement, and the serving transitions.

### *Q. How do you test a WebSocket?*

The FastAPI TestClient supports a WebSocket context manager, so the connection can be opened, a queue transition triggered and the received message asserted, all without a live server. The limitation is that this does not exercise real network behaviour, so fan-out under many concurrent listeners and genuine latency are untested, and I would want a load test before claiming the queue display is reliable at clinic scale.

### *Q. How do you know a fix actually works?*

I wrote the media tests before applying the Grad-CAM fix, and confirmed they failed against the original implementation with 404s, then passed once filename-stem resolution was in place. That matters, because a test written after the fix would only have confirmed current behaviour and would not have demonstrated that the suite can actually detect the defect class. The same applies to the multipart boundary fix and the role-restriction change: each was reproduced, then fixed, then verified.

### *Q. What is not tested?*

Honestly, quite a lot. There are no frontend or browser tests at all, so the interface is verified manually and by lint. The six-stage validation gate has no unit tests with image fixtures, which is the most important gap because it is the least-verified component in the critical path. Queue tests run on SQLite rather than PostgreSQL, so PostgreSQL-specific concurrency behaviour is unexercised. There is no performance or load testing. And no test asserts the absence of duplicate route registration, which is precisely why the duplicated route went unnoticed.

### *Q. What quality gates does the project enforce?*

Three, all currently green. The backend suite runs under pytest with thirty-four passing tests. Frontend lint runs with a zero-warning threshold, so a warning fails the build, and it currently reports none. And the production build must succeed. I would also add a pre-commit hook, because all three of these are currently manual and depend on the developer remembering to run them.

## K. Deployment and operations

### *Q. How do you deploy this?*

Docker Compose, with the stack in the docker directory rather than the repository root. A backend container runs Uvicorn on port 8000, and a frontend container builds the Vite bundle and serves it through Nginx, which terminates TLS, serves the static assets, and falls back to index.html for client-side routes. That history fallback is essential, because without it a browser refreshing on a deep link gets a 404 from Nginx.

### *Q. Why Nginx in front of a containerised frontend?*

Because a Vite build is static files, and serving static files in production is a solved, well-understood problem with excellent caching, compression and TLS behaviour. Nginx also gives a natural single entry point, so the browser sees one origin in production, which removes the CORS surface entirely. It also gives a natural place to enforce rate limiting, which is currently the main security control I have not implemented in the application.

### *Q. What happens when the dev port is busy?*

Vite silently advances to the next port, usually 5174. I found this the hard way, because the frontend was connecting while the browser was on 5174 and nothing appeared to be configured correctly. The application worked, because the CORS allow-list covers 5173, 5174, 5175, their loopback aliases and port 3000, but a hard-coded origin anywhere would have failed. I mention it because it is a common and confusing failure, and the fix is to read the port from the environment rather than hard-coding it.

### *Q. How are configuration and secrets handled?*

Typed settings through pydantic-settings, with a deliberate fail-fast guard: in production the service refuses to start if the secret key is still a known default or the registration keys are missing or too short, raising a RuntimeError rather than booting insecurely. I think failing to start is the correct behaviour, because a service that starts with a default signing key is a service whose tokens anyone can forge. Secrets are never logged and, as you can see from this document, no secret value appears anywhere in the documentation.

## L. Challenges encountered

### *Q. What was the hardest problem you faced?*

The Grad-CAM images. The feature worked, the model worked, and the prediction was correct, but every heatmap returned 404. The cause was a storage design decision: overlays are written beside the original as the stem plus a suffix, with no database column, and the media route looked up the file by matching the stored path against the requested filename. That pattern can never match, because the stored path never contains the overlay's name. The hard part was not the fix, which was small, but diagnosing it, because the error looked like an authorisation problem and the symptom was confined to one sub-feature of a page that otherwise worked.

### *Q. Describe a bug that hid behind a misleading error message.*

Uploads were failing with what looked like a CORS error. The real cause was that five call sites set the multipart Content-Type header manually, which strips the boundary token, so the server rejected the body with 400 Missing boundary in multipart. The browser surfaced a CORS complaint because the preflight and the actual request disagreed, and I spent time inspecting allow_origins before looking at the request headers. The general lesson is to read the server's response body before believing the browser's framing of the failure.

### *Q. What did you get wrong in your first design?*

Two things, both about trusting boundaries I had not verified. First, I initially assumed the frontend guards were sufficient protection, when in fact every authorisation decision has to be made server-side from the verified token. Second, I documented the media service as serving uploads from a static mount with a permissive CORS origin, because that is how the code looked. Auditing it properly showed radiographs were being served statically, which is a genuine privacy problem for medical images, and it is now an authenticated route with per-request ownership verification.

### *Q. How did you verify the model was genuinely working?*

Two independent checks. The startup log states whether the DenseNet-121 loaded and connected, and a public model-status endpoint reports the live connection state, which the frontend uses to gate the submission control so a user cannot attempt analysis against a model that is not loaded. I also confirmed the checkpoint is present at the expected size. The invariant underneath both is that random predictions are disabled, so a healthy-looking interface always reflects a genuinely loaded model.

### *Q. How would you debug this system if it failed in production?*

In layers, from the outside in. Start with the health of the container and the model-status endpoint, since a disconnected model is the failure that disables the core feature. Then check whether the failure is transport, by reading the CORS and status codes on a failing request. Then application, by reading the response body rather than the browser's summary of it. Then data, by checking whether the request reached the database at all. Finally the model, by confirming the checkpoint and the preprocessing contract. Structured logs would be the starting evidence, and I would add request correlation ids, which this project does not yet have and should.

## M. Software engineering practice

### *Q. How did you manage the project?*

Version control with Git throughout, working in feature-scoped commits, and treating the branch as the integration point. I kept schema, backend and frontend concerns separable, and I used the test suite as the gate before considering a change complete. Where I made a decision that traded simplicity for capability, like accepting a static mount and then replacing it, I documented why, because the reasoning is the part that is expensive to reconstruct later.

### *Q. What would you do differently?*

Introduce the test suite earlier. I wrote tests when fixing the Grad-CAM and deletion defects, and they were genuinely useful, but the queue logic and the validation gate were written first and are now the parts I trust least. I would also have introduced migrations from the start, and I would have kept a running decision log, because some of the design reasoning in this project only exists in the changelog section of the documentation.

### *Q. How do you keep code quality up in a codebase this size?*

Three practical measures. A strict lint configuration, where the zero-warning threshold means style regressions are caught by the build rather than by review. A test suite that covers the paths where a regression would be dangerous, namely authentication, authorisation and file handling. And a convention of keeping route handlers thin, with business rules pushed into the service modules, which is why the queue rules are testable without going through HTTP.

### *Q. How would you scale this for a hospital?*

In three moves. Replace SQLite-in-tests with a PostgreSQL test database so concurrency semantics are exercised, and add the unique constraint on doctor, date and token. Move inference behind a worker and return a job identifier, because synchronous inference of a 28 MB model in the request path does not scale past a handful of concurrent uploads. Add request correlation ids and structured logging, because at hospital volume you cannot debug what you cannot trace. After that, horizontal scaling of the API is straightforward since it is stateless.

### *Q. What is the most important thing you learned?*

That the most dangerous failures in a medical system are the quiet ones. A wrong prediction is visible, because a clinician disagrees with it. A fabricated prediction from a model that failed to load, or a radiograph served from a public static path, produces a result that looks entirely normal to everyone who sees it. That is why this project is built around refusing to produce output it cannot justify, and around treating every file as private by default, rather than around maximising model performance.

## N. Backend deep dive: FastAPI, SQLAlchemy and the request lifecycle

### *Q. Walk me through exactly what happens between a request arriving and a response leaving.*

Uvicorn, the ASGI server, receives the HTTP request and hands it to the FastAPI application. Middleware runs first, in reverse registration order, so the security-headers middleware wraps the request, awaits the route, then sets its headers on the way out. FastAPI then resolves the dependency graph: the bearer token through OAuth2PasswordBearer, and the database session through get_db, both declared as Depends and therefore injectable and testable. Pydantic validates and parses the body, coercing types and rejecting anything malformed with 422 before my code runs. The handler executes, and the return value is serialised. The session dependency closes the session. The important design property is that authorisation and database access are declared in the signature, so you can read a route's security requirements from its decorator line.

### *Q. Why use dependency injection instead of a global database session?*

Three reasons. First, testability: a test overrides get_db with a fixture pointing at an isolated database, so no test touches real data, and that is exactly how the suite runs. Second, lifetime: the session is created per request and closed when the request ends, so a connection is never leaked and a transaction cannot accidentally span two users. Third, clarity: a handler that declares token and db as parameters is visibly doing authenticated database work, whereas a module-level global hides that. The trade-off is a little ceremony per handler, which is worth it for anything touching medical records.

### *Q. What is an N+1 query, and does your code have that problem?*

It is when a loop issues one query per row, so ten predictions cause eleven queries instead of one. My code is exposed to it in the places that serialise users with their predictions or their appointments, because SQLAlchemy relationships are lazy by default and will happily emit one query per accessed attribute. It has not been a visible problem because the result sets here are small, a doctor has tens of patients rather than thousands. The fix if it grew would be selectinload or joinedload on the relationship, so the parent query fetches the children eagerly.

### *Q. What is the connection pool, and what are its defaults?*

SQLAlchemy maintains a pool of open connections so that establishing a TCP connection and authenticating to PostgreSQL, which is comparatively expensive, is not paid on every request. The QueuePool default keeps five connections and allows ten overflow, recycling after about half an hour of idle time. With uvicorn workers that multiplies, because each worker process holds its own pool, so the real ceiling is the pool size times the worker count. Under a clinic load that is plenty, but it is the first thing I would check if a deployment started timing out under load.

### *Q. Why is the predict route async when the model runs on the CPU?*

The route is async so that awaiting the file read does not block the event loop, which matters because a radiograph upload can be several megabytes and there are other users on the queue display. The model inference itself is synchronous CPU work, so it does block the loop while it runs, and that is a real limitation. The honest fix for scale is to move inference to a process or task queue and return a job identifier the client polls or receives over the WebSocket, rather than holding a request open for the duration of a forward pass.

### *Q. What is the difference between 400, 401, 403, 404, 413 and 422, and where does your code use them?*

400 is a malformed or unacceptable request, which I use for a failed image validation and an invalid access code. 401 means unauthenticated, so no valid token was presented, and it is what the auth dependency raises. 403 means authenticated but not permitted, which is what a patient hitting a doctor-only route receives. 404 is used for a missing record and, deliberately, for a media file the caller is not entitled to, so that an unauthorised request cannot be distinguished from a missing one and cannot be used to probe what exists. 413 is the payload-too-large response for the 10 MB ceiling. 422 is Pydantic's validation failure, raised before my handler runs.

### *Q. Why return 404 instead of 403 when a user is not allowed to see an image?*

Deliberately. If the server returned 403 for a radiograph belonging to another patient, then the response itself would confirm the file exists, so an attacker could enumerate valid filenames and learn how many records exist and roughly when they were created. Returning 404 makes an unauthorised request indistinguishable from a genuinely missing one, so the endpoint leaks nothing. This is the general principle that identifiers should not be guessable and that existence should not be confirmable without authorisation.

### *Q. How does the media route decide whether the caller may see the file?*

It re-derives the identity from the token, resolves the prediction row that owns the requested filename by stem, and then applies an access predicate that permits the prediction's owner, the treating doctor, and administrators. The check is per request rather than baked into the URL, which is the point: possession of a filename is not authorisation. Admin access is deliberate so a moderator can investigate an abuse report, and it is the one case where a non-owner can view a radiograph.

### *Q. What does secure_filename do, and why do you not use it?*

It is a Starlette helper that strips directory components and unsafe characters from a user-supplied filename, which is the standard defence against path traversal in an upload handler. In the predict route I do not need it, because I discard the supplied name entirely and generate a UUID filename myself, so there is no attacker-controlled string anywhere near the path join. That is stronger than sanitising untrusted input, since sanitising is a denylist and generating is not. The media route still enforces basenames and verifies the resolved absolute path stays inside the uploads directory.

### *Q. How do you handle a database error in a handler?*

Poorly in some places, and I would rather say so. There is no global exception handler, so an unhandled database error becomes a 500 with FastAPI's generic body, which is acceptable but not helpful. What I do have is the session dependency, so a rollback on failure is a single place rather than scattered try and except blocks. The improvement I would make is a global exception handler that logs the traceback with a correlation id and returns a generic message, so internal detail is never leaked to a client while the log keeps the evidence.

### *Q. How is configuration read, and what happens if it is missing?*

Settings are typed through pydantic-settings, so a missing or malformed variable fails at import with a clear message rather than becoming None and failing later somewhere unrelated. On top of that there is an explicit production guard: the service raises a RuntimeError at startup if the signing key is still a known default or the registration keys are absent or too short. I consider failing to start the correct behaviour, because a service that boots with a default signing key is a service whose tokens anybody can forge, and that failure would be silent.

### *Q. What is the WebSocket lifecycle, and what happens on disconnect?*

On connect the handler accepts the socket and registers it with a connection manager, which keeps a collection of active connections so a state change can be broadcast. Because a WebSocket has no request scope, the handler pulls the session factory from application state and opens its own short-lived sessions, which it closes explicitly; sharing one session across connections would be unsafe under concurrency. On disconnect the connection is removed from the manager, and broadcast iterates only the live set, so a closed socket cannot raise and break delivery to everyone else.

### *Q. What happens if a client is offline when a queue event fires?*

The broadcast is missed, which is the fundamental weakness of push-only updates. That is precisely why the client treats the WebSocket payload as a signal rather than as data: on receiving any message, including a reconnect notification, it re-fetches the authoritative queue state from the API. So a missed event costs a slightly stale display rather than a permanently wrong one. The alternative of replaying an event log would be more robust and considerably more complex.

### *Q. How do you keep a long-lived connection from failing silently?*

The main risks are a network drop with no reconnect, and a server restart. I handle reconnection on the client with a retry, and the broadcast is a prompt to re-fetch rather than the source of truth, so even a connection that never recovers degrades to manual refresh rather than a wrong queue. A production version would add a heartbeat, because a TCP connection through a NAT or proxy can appear open long after it is dead, and without a ping the display would simply stop updating without any visible error.

### *Q. Why two routers instead of one?*

Separation of concern with a practical benefit. The admin router mounts at the same /api prefix but lives in its own module, so the administrative surface can be read, reviewed and restricted in isolation, and it would be straightforward to move behind a different authorisation layer or a separate origin later. The cost is one extra import and include_router in main.py. The prefix is declared per include_router call rather than per router, which keeps the mount point visible in one place.

## O. Frontend deep dive: React, Vite and the browser

### *Q. How is state managed across the app, and why no Redux or Zustand?*

State is deliberately local: around two hundred useState calls, most of them inside a single page component, with data fetched per page. The only genuinely shared state is identity and role, which lives in an AuthContext, and the theme, which lives in a ThemeContext. I avoided a global store because there is very little state that more than one component needs, and a global store would add a dependency, an action vocabulary and a mental model without removing any duplication I actually had. If the app grew screens that needed the same server data, I would reach for a query cache, not a hand-rolled store.

### *Q. Why lazy-load most pages, and which ones are not?*

Thirteen pages are loaded through React.lazy with a Suspense boundary, and only Home, Login and Register are in the initial bundle. The reasoning is that the landing page and the two routes an unauthenticated visitor needs must be available immediately, while DoctorDashboard alone is over 1800 lines and Patients and Profile are larger still, so shipping them to a visitor who will never open them is wasted bandwidth. The effect is real: the initial bundle is a fraction of the total, and the heavy clinical screens parse on demand.

### *Q. What is the trade-off of lazy loading that you accepted?*

One extra round trip on first navigation to each route, because the chunk has to be fetched before it renders. That is why the Suspense fallback is a deliberate page-level loading state rather than nothing, so the user sees a consistent frame instead of a flash of empty content. The second cost is that a chunk which fails to load leaves the user on a spinner, so a chunk-level error boundary would be the natural companion to the fallback and I do not have one.

### *Q. Explain useEffect and its dependency array.*

useEffect runs a side effect after render, and the dependency array decides when. With an empty array it runs once after mount and cleans up on unmount, which is what I use for one-time setup like opening a WebSocket or reading a stored preference. With dependencies it re-runs when any of them change by identity, which is how a fetch re-issues when a filter or an identifier changes. The rule I follow is that the array must list everything the effect reads, because an omitted dependency produces a stale closure that is a frequent source of subtle bugs.

### *Q. What is a stale closure, and where did you hit it?*

It is when a callback captures an old copy of a state variable and keeps using it after the state has moved on, usually inside a timer or an event handler registered once. The classic symptom is a counter that only ever increments to one. The fixes are the functional setState form, where the previous value is passed in rather than captured, and including the value in the dependency array. I also use useCallback for handlers that are passed to memoised children, though I will be honest that with only two uses in the codebase I have been inconsistent about it.

### *Q. Why is useRef used so much, and when is it the wrong choice?*

Around forty-one uses, mostly for two things: holding a reference to an input element so a label click can focus it or a custom upload box can open the native picker, and holding mutable values that must not trigger a re-render, such as a timer id or a socket instance. The distinction that matters is that changing a ref does not re-render, so it is correct for imperative handles and wrong for anything the interface displays, which belongs in state. Using a ref for display state is the most common React mistake I see.

### *Q. How does the dark theme actually work?*

ThemeContext holds the mode, and it initialises from localStorage if a preference is stored, otherwise from the operating system via the prefers-color-scheme media query, so a first-time visitor gets the theme they already have set elsewhere. An effect then toggles a dark class on the document element and persists the choice. Because the class lives on the root element rather than in component state, Tailwind's dark variants apply everywhere without prop drilling, which is why it is done at the document level.

### *Q. Why a custom event for session expiry instead of a redirect inside the interceptor?*

Because the axios layer should not own navigation. It clears the stored session and dispatches a pulmo-ai:session-expired DOM event; AuthContext listens for it and updates React state, which then drives the redirect through the router. Two benefits: the redirect is a client-side transition rather than a full page load, and the API layer stays free of routing concerns so it could be reused by a non-router consumer. The event is also decoupled, so multiple listeners can react, for example to close a WebSocket or clear cached data.

### *Q. How do you keep a token from being attached to a public request?*

The request interceptor runs an isPublicAuthRequest check against the login, register and password paths and returns early, so no Authorization header is added there. That is not only tidiness, it is correctness: a stale token sent with a login request would be meaningless, and a 401 returned from a public auth route would trigger the expiry handler and bounce the user away from the form they are trying to submit. The same guard runs in the response interceptor for the same reason.

### *Q. How are protected images rendered, given that an img tag cannot send a header?*

There is a getProtectedImageUrl helper. It normalises whatever path the API returned, so a stored value like uploads/radiology and a bare filename both resolve to the authenticated media route rather than a static path, it leaves genuinely absolute URLs and profile avatars alone, and then it appends the token from storage as a query parameter. It also supports an optional version argument that becomes a cache-busting query, which is how a re-uploaded file at the same path is forced to refetch instead of showing a cached copy.

### *Q. What does that cache-busting parameter actually solve?*

Browser caching of an image URL. If a filename were reused, the browser would serve the stale image from cache and the interface would appear to show the wrong radiograph, which in a clinical context is alarming rather than merely annoying. Appending a version or timestamp changes the URL, so the cache key changes. It is a pragmatic fix; the cleaner approach is immutable content-addressed filenames, which the UUID naming already gives us for predictions but not for avatars.

### *Q. How do you generate the PDF, and what are its limitations?*

html2pdf.js runs in the browser and composes the result view into an A4 document that the user downloads. The advantage is that no report data is sent to a third-party service, which matters for medical content. The limitations are real: it rasterises through the browser, so the text is not selectable or searchable, pagination is approximate, and a very long history could produce an awkward break. For an official archival clinical document I would render it server-side, where I control pagination and get real text.

### *Q. How is form state and validation handled?*

Forms are controlled, with each input bound to state and validated on submit rather than on every keystroke, because validating per keystroke on a password field is hostile. The backend is the real validator, since it must validate regardless of what the client does, and the frontend mirrors the important rules to give fast feedback. Errors from the server are extracted centrally by a getErrorMessage helper that understands both a plain string detail and the array-of-objects shape that Pydantic returns for validation errors, so components do not each reimplement that parsing.

### *Q. How is the WebSocket consumed on the frontend?*

A hook opens the socket when the component mounts and closes it on unmount, and on any message it re-fetches the authoritative state rather than patching local state from the payload. That deliberately pessimistic choice costs one extra request per event and removes a whole class of bug where two clients disagree because one applied a partial update. The cleanup function closing the socket on unmount matters, because a socket left open leaks and keeps firing setState on an unmounted component.

### *Q. What is the difference between a controlled and an uncontrolled input?*

A controlled input takes its value from React state, so React is the single source of truth and you can validate or transform it. An uncontrolled input keeps its own DOM state and React only reads it via a ref, which is less re-rendering and is the right choice for something like a file input or a large text area that does not need live validation. I use controlled inputs for the clinical forms because they need validation, and refs for the upload box and for reading values once at submit.

### *Q. How do you handle a list that can grow, and why do keys matter?*

Lists are rendered with map, and every item gets a stable key. Keys matter because React uses them to decide whether an element in the new list corresponds to an element in the old one. With an index as the key, inserting or deleting an item in the middle makes React reuse the wrong component instances, so state and focus follow the position rather than the record, which produces visible bugs such as the wrong input value appearing in a row. For data with a real identifier I use it, and I only fall back to index for static lists that never reorder.

### *Q. What is the difference between Vite and Create React App?*

Vite serves the source over native ES modules during development, so startup is near instant and hot module replacement updates only the modules that changed, whereas CRA bundles with webpack and rebuilds. For production Vite uses Rollup and code splitting, which is what makes the per-route lazy chunks possible. CRA is now deprecated, so for a new project it is not really a choice. The practical effect for me was iteration speed on an interface with several thousand lines.

### *Q. How do environment variables reach the client, and what is the risk?*

Vite exposes only variables prefixed with VITE_ and inlines them into the bundle at build time, so anything the client reads is visible to anyone who opens the developer tools. That is why no secret is ever a Vite variable; the JWT signing key and the database URL exist only on the backend. The API origin is a VITE_API_URL, which is a non-secret address, and in production Nginx serves the app behind one origin so the value is often unnecessary.

### *Q. How do you make a page usable on a phone?*

Tailwind drives a mobile-first layout, and the components collapse to a single column at small widths, with the doctor directory becoming a two-column grid on mobile and three on desktop. Beyond layout, the details that matter on a phone are touch target size, the bottom navigation collapsing to a drawer, and file inputs that open the camera roll directly. I also had to stop action labels wrapping, because a two-line button row in a results card misaligned the whole row.

## P. Frontend and backend integration

### *Q. How exactly are the frontend and the backend connected? Describe the actual mechanism.*

There is no development proxy, which surprises people who assume Vite proxies the API. In development the Vite server serves the React app on port 5173 and the browser makes real cross-origin requests straight to FastAPI on port 8000. A single shared Axios instance in services/api.js holds the base URL, built as VITE_API_URL with /api appended, and every relative path in the app is written against that base. The backend mounts both routers under a single /api prefix and enables CORS for the dev origins on 5173, 5174, 5175 and 3000, plus the loopback equivalents, with credentials allowed. In production the built static files are served alongside the API, so the deployment is same-origin and the CORS question mostly disappears.

### *Q. Why call the backend directly instead of configuring a Vite dev proxy?*

The main reason is that the auth token is deliberately not a cookie. It is a bearer token in localStorage that the Axios interceptor attaches, so there is nothing for a proxy to forward and nothing for it to protect. A proxy earns its keep when the browser must present a cookie automatically, or when you need to collapse two services onto one origin during development. Neither applies here, and calling the backend directly means the code path in development is the same one that runs in production. The cost is that CORS has to be configured correctly and every call site has to know the absolute base URL, which is where a few of the inconsistencies in the next few answers come from.

### *Q. How does the frontend authenticate its requests, and how does it handle an expired token?*

The request interceptor reads the token from localStorage and sets an Authorization header with the Bearer scheme, but it deliberately skips that for the public auth routes, because attaching a stale token to a login attempt is what triggers spurious 401 loops. On a 401 from a protected route the response interceptor clears the stored session and dispatches a custom DOM event, pulmo-ai:session-expired. AuthContext listens for that event and performs the router transition to the login page. The reason it is an event rather than a direct redirect inside api.js is that the service layer has no business knowing about routing; keeping navigation in the context means the two concerns stay separate and the redirect happens exactly once, at one place.

### *Q. Why store the JWT in localStorage when an HttpOnly cookie would be safer?*

An HttpOnly cookie is genuinely the stronger choice against token theft, because script cannot read it, and it comes with SameSite protection against CSRF for free. I did not use it here, and the reason is that the API is consumed as a token-based service, where the client is expected to hold and present the credential itself. The honest characterisation of the trade-off is that localStorage is readable by any script running on the page, so an XSS bug becomes a token theft bug. I mitigate that side rather than the storage choice, by sanitising what is persisted, stripping token-shaped and password-shaped keys out of the stored user object so a careless spread of the login response cannot leak a credential into storage, and by relying on React's escaping rather than dangerouslySetInnerHTML for rendering anything I did not write.

### *Q. How does the frontend turn a failed request into a message a human can act on?*

FastAPI returns errors in several shapes, so a single naive read of the body is not enough. A 422 carries an array of Pydantic validation objects with loc, msg and type, where the useful part is usually the field name and the message together. A 400 or 401 from my own handlers returns detail as a plain string. So there is a shared helper that inspects the payload, flattens the Pydantic array into readable field-plus-reason strings, accepts a bare string detail, and also checks a message key before falling back to the status text. Every page reports through that one helper, which is why an error reads the same way on the booking page as on the dashboard.

### *Q. How is a chest X-ray actually uploaded, and what was the subtle bug there?*

The file is put into a FormData object with the model and the image as parts, and the request goes out as multipart form data. The subtlety is that you must not set the Content-Type header yourself. If the client sets application/json, or sets multipart without a boundary parameter, the browser does not append the boundary and the server cannot parse the body, so the request fails in a way that looks like a server bug rather than a client one. The fix is to delete the explicit Content-Type for multipart calls and let the browser generate the header with the boundary. I had that wrong in five separate places and fixed them together.

### *Q. How does the frontend display a radiograph it is not authorised to fetch directly?*

This is the interesting constraint, because an img tag cannot carry an Authorization header. You cannot put the token in a header on a subresource request, and embedding the token in the URL of a plain image is exactly the pattern that leaks tokens through logs and referrers. The solution in this system is a helper that builds the media URL with the token as a query parameter, which the media route validates before streaming the bytes. It works, and it is a known compromise rather than a clean design. A better answer would be a short-lived signed URL or a blob fetched with fetch and rendered from an object URL, which is what I would move to. The same helper appends a version parameter so that a replaced image is not served from cache under its old name.

### *Q. Some pages call axios directly instead of using the shared instance. Why?*

That is a genuine inconsistency and not a deliberate design. Sixteen files go through the shared instance. GuestBooking calls raw axios, but only for the three endpoints that are public anyway, the doctor directory, the queue window and the registration request, so it is harmless. The problem is TodayTokens and DoctorDashboard, which hand-roll roughly twenty calls and attach the Authorization header themselves. They work, but they duplicate the token logic in twenty places and, more importantly, they bypass the centralised 401 handling, so when the session expires on those two screens the user gets an inline error instead of being returned to the login page. That is the kind of divergence that only shows up in production, and migrating them to the shared instance is on my list.

### *Q. How is a file upload kept safe from path traversal and from serving someone else's scan?*

Two separate defences. The stored filename is a generated UUID with the original extension, not the client-supplied name, so a traversal sequence in the filename has nothing to attach to because the client name is discarded entirely. And authorisation is enforced on read as well as on write: the media route resolves the requested path, checks it is inside the upload directory, verifies the requester is either the owner or an authenticated doctor, and only then streams the file. Deletion follows the same ownership check, and the tests cover the traversal attempt and the cross-patient access attempt explicitly rather than assuming them.

### *Q. How do live queue updates get from the backend to the browser?*

Polling would have been simpler, so the WebSocket is a deliberate trade. FastAPI exposes a /api/ws/queue endpoint backed by a connection manager that holds the live sockets and broadcasts whenever a token is called, served or moved. The frontend subscribes on the waiting-room and doctor screens and updates the queue in place instead of re-fetching. The cost is that connection lifecycle becomes my responsibility, since I have to handle disconnects, avoid broadcasting into a dead socket, and deal with the half-open connection case where a client vanishes without a clean close. For a clinic with a handful of screens, polling every few seconds would honestly be the more robust choice, and I would keep the WebSocket only if the latency difference mattered.

### *Q. How do you stop the frontend and the backend from drifting out of sync?*

Three things. The backend owns Pydantic schemas, so the response shape is defined once on the server and the frontend consumes it rather than inventing it. The frontend gates its screening screen on a model-status endpoint, so if the model fails to load the interface degrades explicitly instead of offering an upload that will always error. And the route surface is documented and verified, all fifty distinct endpoints with their methods and line numbers, so a renamed path is a detectable difference rather than a silent 404 found by a user. What I would add is generated client types from the OpenAPI schema, so a field rename fails at build time instead of at runtime.

### *Q. What happens in the frontend when the backend is unreachable?*

Axios rejects with a network error rather than an HTTP status, so the error-shape helper I described will not find a detail field and falls through to the status text, which in that case is not very informative. The screens that depend on the API surface that failure through their own error state rather than crashing, but the weakness is real: there is no route-level error boundary in the React tree, so an unhandled render-time exception blanks the page instead of showing a recoverable message. A boundary per route is the fix, and it is on the list along with migrating those two raw-axios pages.

### *Q. Your write-up names PostgreSQL, but a reviewer noticed SQLite. Which is it?*

Both, and the distinction is deliberate. Production and the working environment run PostgreSQL through SQLAlchemy, configured from a DATABASE_URL setting, and the queue allocation depends on real row-level locking behaviour that SQLite does not reproduce faithfully. The test suite deliberately runs on in-memory SQLite with a static pool so that tests are fast, hermetic and need no server, and I state that limitation in the test documentation rather than hiding it, because it means the concurrency tests do not exercise PostgreSQL locking semantics. The application schema itself stays portable because it is expressed through the ORM and does not reach for engine-specific features.

## Q. Experimental results and the evidence behind the claims

### *Q. State your headline result and tell me exactly how you measured it.*

The proposed model is a DenseNet121 fine-tuned from ImageNet weights for three-class chest radiograph triage, Normal, Tuberculosis and Pneumonia. The reported figure is 96.65 percent accuracy with 96.50 precision, 96.60 recall and a 96.55 F1 score, measured on a held-out test set of 4,500 radiographs, 1,500 per class, that was never used for training or for hyperparameter selection. Three architectures were trained under one protocol so the comparison is fair: same 224 by 224 input, same CLAHE preprocessing, same ImageNet normalisation, same Adam and cross-entropy setup, same 50 epochs. ResNet50 reached 94.10 percent and VGG16 91.25 percent, so the proposed backbone leads by 2.55 and 5.40 points respectively. I would also immediately say that those headline percentages should be read alongside the confusion matrix rather than on their own, which is the next question.

### *Q. Your confusion matrix and your accuracy table do not agree. Which one is correct?*

They cannot both be right, and I would rather say so than pick the flattering one. Adding the diagonal of the published matrix gives 1,450 plus 1,420 plus 1,420, which is 4,290 correct out of 4,500, and that is 95.33 percent, not the 96.65 percent quoted in the comparison table. So the matrix is the more reliable artefact, because it is the raw per-cell count and every other metric can be recomputed from it, whereas an accuracy printed in a summary table can be a transcription from a different run. My reading is that the summary row and the matrix came from two different evaluation passes and were merged when the chapter was assembled. The honest position is that the defensible claim is the 95.33 percent that the matrix supports, and the 96.65 should be treated as unverified until the evaluation script is re-run and both are regenerated from one artefact. The ranking against ResNet50 and VGG16 is unaffected, because a uniform error would shift all three models together.

### *Q. Compute the accuracy from your own confusion matrix, live.*

Rows are actual class, columns are predicted. The diagonal is Normal 1,450, Tuberculosis 1,420, Pneumonia 1,420. That is 4,290. The total is 4,500, because each row sums to 1,500. So accuracy is 4,290 over 4,500, which is 0.9533, or 95.33 percent. If you are being asked for precision, the column sums are 1,520 for Normal, 1,500 for Tuberculosis and 1,480 for Pneumonia, so Normal precision is 1,450 over 1,520 which is 95.39 percent, Tuberculosis is 1,420 over 1,500 which is 94.67, and Pneumonia is 1,420 over 1,480 which is 95.95. Macro-averaging those gives 95.34 precision and 95.33 recall.

### *Q. Your class-wise precision column also disagrees with the matrix. Does that change your conclusions?*

It changes the precision figures but not the shape of the conclusion. The recall column does reconcile, almost exactly: 96.70 against a derived 96.67 for Normal, 94.70 against 94.67 for Tuberculosis, and 94.80 against 94.67 for Pneumonia. Precision is where it diverges, by 2.11 points on Normal and 1.25 points on Pneumonia in opposite directions. Since precision depends on how many cases were predicted into that class, a divergence there usually means the table and the matrix were produced from different runs rather than that a calculation is wrong. The conclusion I would defend is the ranking of the three backbones and the finding that the two disease classes are confused with each other far more than with Normal. The specific per-class precision decimals I would not defend until recomputed.

### *Q. Why is accuracy a poor headline metric for this problem, and why did you macro-average?*

Because the test set is deliberately balanced, 1,500 per class, which is the right choice for training but hides the thing that matters clinically. In a real clinic most films are normal, so accuracy on a balanced set is not an estimate of real-world performance at all; a model could score 96 percent here and behave much worse on a population with 80 percent normals. I macro-average, which is to say I compute precision, recall and F1 independently for each of the three classes and then take the arithmetic mean, so a class with few examples cannot be hidden inside an aggregate. For a screening tool the metric I would lead with is per-class recall on the two disease classes, because a missed active case is the expensive error, not a misfiled normal.

### *Q. What is ROC-AUC, why is it per class, and what were your values?*

ROC-AUC is the area under the curve of true-positive rate against false-positive rate as you sweep the decision threshold. It is 0.5 for chance and 1 for perfect, and it is threshold-independent, which matters here because the operating point of a screening tool should be chosen deliberately rather than inherited from whatever threshold argmax happens to give. It is reported per class using the one-versus-rest formulation, so each disease is treated as the positive class against everything else. The reported values are 0.992 for Normal, 0.985 for Tuberculosis and 0.988 for Pneumonia. Tuberculosis has the lowest, which is consistent with it being the class most often confused with the other disease rather than with Normal.

### *Q. You claim a false-negative rate of 2.33 percent for active disease. Is that good enough for screening?*

It is better than a typical manual error rate and it is not good enough to be a gate. Breaking it down: 40 Tuberculosis films were called Normal and 30 Pneumonia films were called Normal, which is 70 of 3,000 active cases, or 2.33 percent. For context that means roughly one in forty-three active cases is missed on a single frontal film. Two things make that acceptable for triage and unacceptable for diagnosis. It is acceptable because the output is advisory, the film still goes to a clinician, and the interface never presents the result as a finding. It is unacceptable as a sole gate because two thirds of misses are the two disease classes being mistaken for each other rather than for Normal, and a system that cannot separate them should not be trusted to exclude either. The mitigation is a clinical rule rather than a model change: any radiographic indication of TB overrides the model output.

### *Q. Your largest error is TB confused with Pneumonia. Is that clinically acceptable?*

The counts are 40 Tuberculosis films called Pneumonia and 50 Pneumonia films called Tuberculosis, which together is 90 of the 210 total errors, so it is the dominant failure mode by a wide margin. I consider it the most clinically defensible error in the matrix, and the argument is anatomical rather than statistical: both diseases produce alveolar consolidation and increased opacity, and on a single frontal projection they frequently overlap. A radiologist reading a film with a dense upper-lobe consolidation faces the same ambiguity. That does not excuse it, because the two have different urgency and different management, TB requiring isolation and a specific drug regimen. So I would not present this as a solved problem. The honest framing is that this system is good at separating disease from normal and materially weaker at separating the two diseases from each other, and that a lateral view plus clinical history is what would resolve it, neither of which this system sees.

### *Q. You benchmarked 500 simulated requests. Is a latency measurement on synthetic load valid?*

It is a valid measurement of the system's cost, and it is not evidence about production behaviour. The breakdown is 35 milliseconds for multipart ingestion and disk write, 42 for CLAHE and normalisation, 118 for the DenseNet forward pass on an Intel Core i7 CPU, and 50 for report and database write, totalling 245. The arithmetic checks out. What 500 simulated requests cannot tell you is behaviour under concurrency: a single figure of 245 milliseconds says nothing about the 95th percentile, nothing about two doctors uploading at once, and nothing about whether the process saturates. Since the forward pass is synchronous and CPU-bound, it holds a worker for 118 milliseconds, so throughput is bounded by the worker pool, and that is exactly the thing a sequential benchmark is blind to. I would report a percentile distribution under concurrent load before claiming the number holds up in a clinic.

### *Q. How do you know your test set is genuinely independent, and how do you know there is no leakage?*

Stratified splitting with a fixed hold-out, and the test partition was touched once, at the end, after early stopping had already selected the checkpoint on validation loss. The mechanisms I used are that the split was stratified per class, that the same partition was reused across all three backbones so no model got an easier test set, and that checkpoint selection used validation loss rather than test accuracy. But I should be honest about the real risk here, which is not train-test leakage, it is source leakage. The images were pooled from four public archives, and those archives contain studies where the same patient appears in multiple frames. If a patient appears once in training and once in test, the model is being tested on memorisation rather than generalisation, and a split at the image level cannot detect that. Grouping by patient identity is the fix, and whether the public archives expose identity consistently is something I did not verify.

### *Q. What is the single biggest threat to the validity of these results?*

The dataset provenance, and it is worth being blunt about why. The classes are assembled from four different public archives, and in chest radiography the scanner, the hospital protocol and the image processing chain leave strong fingerprints. A model can reach high accuracy by learning which archive a film came from rather than what is in the lung field, because TB-heavy and Pneumonia-heavy subsets often came from different collections. The result would look excellent and generalise to nothing. The way to test it is to hold out an entire archive rather than a random sample, and report performance on that unseen source. I did not do that, and it is the first thing I would fix. The second is the internal inconsistency in the reported metrics, which is a reporting failure rather than a modelling one, and the third is the absence of any local or prospective validation, since every image is retrospective and public.

### *Q. If you could re-run one experiment, which would you choose and what would you expect to happen?*

Leave-one-archive-out evaluation, for the reason above. I expect the number to fall, and I would rather know that now. If accuracy on a completely unseen source dropped to the low eighties, that would not invalidate the project, it would relocate the claim from a diagnostic tool to a research prototype that needs domain adaptation, and it would tell me which augmentation and normalisation choices actually matter. The current reported figure is a best case measured under conditions I constructed, and I cannot currently say how far it is from a best case.

## R. Literature review, research gaps and justification

### *Q. Which research gaps did you identify, and how does each one map onto a design decision?*

I identified five, and I deliberately mapped each to a concrete choice rather than leaving them as a literature summary. Gap one was the local versus global feature trade-off, which is why I chose a densely connected backbone, since dense concatenation propagates fine local features through depth while the deeper layers add context. Gap two was parameter-heavy footprints blocking edge deployment, which is why the target is roughly 7 million parameters and CPU inference rather than a 138 million parameter VGG16. Gap three was the field's fixation on binary tasks, which is why this is three-class and can separate TB from Pneumonia rather than only disease from normal. Gap four was models published without software, which is the entire second half of this project. Gap five was the black-box objection, which is why Grad-CAM overlays are served to the clinician rather than omitted.

### *Q. Several studies you cite report higher accuracy than you. Why not use their models instead?*

That is fair, and I have thought about it. The cited results include 99.52 percent for a GhostNet with MobileViT hybrid, 99.20 for DenseNet201 with CLAHE, and 92.31 for ConvNeXt-Tiny on a five-class set. Two of those beat my headline figure. The reasons I did not simply adopt them are that the higher numbers come with costs that conflict with my constraints, and that the comparison is not like for like. Mahamud's 201-layer DenseNet201 is explicitly noted as requiring substantial GPU memory, which is the opposite of a CPU deployment target. Owda's work is described as primarily binary disease-pair classification, so it is not solving the multi-class problem I set. And the near-99 percent figures generally come from single-source or curated subsets, whereas I pool four archives, which if anything makes my task harder. The uncomfortable truth is that without a shared evaluation protocol these numbers are not directly comparable, and part of the honest answer is that nobody can settle it without a common test set.

### *Q. Why did you not use a Vision Transformer, given that you identified global context as a gap?*

Two reasons, and the second is the honest one. The first is that ViTs are data hungry, they were one of your own cited findings, and I had a fixed and modest budget of radiographs, so starting from ImageNet weights and expecting them to beat a well-tuned CNN in that regime was optimistic. The second is that your own literature notes slow convergence for transformers on this data, which is an observation I would want to reproduce before adopting one. Testing a Swin or ViT small backbone on the same split is the obvious next experiment and it is listed as future work. I would add a caveat to my own gap statement: having identified the local versus global trade-off as a gap, I addressed it with an architecture that is strong on the local side, not on both.

### *Q. How is your work different from CheXNet, which also used DenseNet121 on chest radiographs?*

The backbone is the same, and I should not claim architectural novelty, because there is none. The differences are in the task and the deliverable. CheXNet-style work frames chest radiograph classification as multi-label, tagging each of fourteen findings independently, because a single film can legitimately show several. Mine is mutually exclusive three-class triage, which is a deliberate narrowing: it matches a triage question, where the system must name one thing to act on, and it is what lets me show a single ranked distribution to a clinician. The second difference is the delivery. That line of work reports accuracy and AUC on curated benchmark subsets. This project is a working clinical application with authenticated access, an auditable prediction history, an OPD queue, and Grad-CAM served under access control, and that integration layer is the part that the literature almost entirely omits and the part your gap four describes.

### *Q. Why is binary classification genuinely insufficient for a general clinic?*

Because the decision a general practitioner faces is rarely binary. The realistic question at a clinic without a radiologist is not is-this-normal, it is what should I do next with this patient. A TB-versus-normal model forces the clinician to discard the pneumonia possibility, and a pneumonia-versus-normal model forces them to discard TB, which is the one that needs isolation and a public-health notification. Both abstractions also mis-state the prevalence problem, because collapsing to two classes makes normal look like a minority class and changes the precision a model can achieve. Three classes let the system say which disease it thinks it is, which is what makes the output actionable. The cost is that TB and pneumonia become mutually exclusive in a way they are not on a real film, and that is precisely the confusion my matrix shows.

### *Q. You claim to fill the gap between AI models and clinical software. Is a student project a genuine contribution to that gap?*

I would not frame it as a research contribution to the gap, and I would be cautious if an examiner pushed me there. What it is, honestly, is a worked example. The gap is real and widely acknowledged, but a project demonstrating that the integration is feasible is not the same as resolving it, because the hard parts of clinical adoption, validation, regulatory approval and workflow fit, are precisely the parts a student project cannot touch. The defensible claim is narrower: this shows a working end-to-end path from upload to audited clinical record, and it documents the engineering decisions and their costs along that path, which is useful to someone who then has to do it with real clinical oversight. It is a feasibility demonstration, and I would label it that way.

### *Q. Make the parameter-efficiency argument quantitatively. Why is 7 million parameters the right target?*

The comparison in the literature table is VGG16 at about 138 million parameters, ResNet50 at 25.6 million, and DenseNet121 at roughly 7 to 8 million, and the state dict is 28.4 megabytes. The argument has three parts. Memory and load time scale with parameter count, so a 138 million parameter model is a 20-fold larger artefact to ship and to hold resident. More importantly, parameter count is a proxy for FLOPs and therefore for latency, and latency is my binding constraint, since the target is under 120 milliseconds on a CPU rather than a GPU. The measured 118 millisecond forward pass is what makes the whole triage argument viable, because if inference took two seconds the queue would be worse than the radiologist backlog I set out to fix. The honest caveat is that parameter count is a rough proxy and the measurement, not the ratio, is the evidence.

### *Q. What would a reviewer legitimately attack you for in the literature review?*

Three things, and I would rather name them. First, the review is selective: I cite the strongest results to motivate the design, which is standard practice but is a form of selective citation, and a reviewer would note that no critical reanalysis of those papers is attempted. Second, hyperparameter selection is described as based on preliminary experimentation without reporting what was searched, so the tuning effort is not auditable and the comparison between backbones could be confounded by unequal tuning. Third, the field moves quickly and the most recent entries are from 2024 to 2026, so a reviewer would reasonably ask whether the gap statement is still current. None of those undermine the implementation, but they do mean the review is a justification rather than a systematic study, and I would not claim it is the latter.

## S. Dataset, preprocessing and training protocol

### *Q. How many images did you train on? Your write-up gives several different numbers.*

My write-up is inconsistent on this and I should own it. Chapter three states 30,000 images at 10,000 per class, then a single table on the facing page states 110,000 as 50,000 Normal plus 30,000 TB plus 30,000 Pneumonia, chapter four states 15,000 at 5,000 per class, and the conclusion returns to 15,000. The defensible figure is 30,000, and here is why. It is the value that is arithmetically consistent with the results: a 70:15:15 stratified split of 30,000 gives exactly 21,000 training, 4,500 validation and 4,500 test, and the reported confusion matrix is explicitly over 4,500 test images. The 110,000 table cannot be right on its own terms, because seventy percent of 110,000 is 77,000, not the 21,000 stated beside it. The 15,000 figure in chapter four is the outlier, because its 80:10:10 split would give 1,500 test images, and I did not evaluate on 1,500. So the results section is self-consistent with 30,000, and the two other figures are transcription errors that need to be corrected to 30,000 throughout.

### *Q. Why does one chapter say seventy fifteen fifteen and another say eighty ten ten?*

Because I changed the split during the work and did not propagate the change, which is a documentation failure rather than a methodological one. The final configuration that produced every number in the results chapter is 70:15:15 on 30,000 images, giving 21,000 for training, 4,500 for validation and 4,500 for the hold-out test set. The 80:10:10 split described in chapter four corresponds to the 15,000-image figure, which is an earlier, smaller configuration. The reason to prefer the 70:15:15 split, and the reason I moved to it, is that fifteen percent for validation gives 1,500 images per class for checkpoint selection, which is enough for the validation-loss curve to be stable, and fifteen percent held out gives 4,500 test images, which is what makes the per-class figures in the results table reasonably precise. A ten percent test split would have given 500 per class, and the confidence intervals on those numbers would be wide enough to make the backbone comparison unreliable.

### *Q. Why stratify the split, and what would break without it?*

Stratification guarantees each subset preserves the class proportions, so the test set has 1,500 of each class rather than whatever the random draw produced. Without it you risk a test set with proportionally fewer disease cases, and since accuracy is sensitive to class balance, the headline number becomes a function of the draw rather than of the model. On a smaller repository the risk is severe, because random assignment can leave a class with very few test examples, and a per-class recall computed on twenty examples is not a measurement. Stratification does not eliminate variance, it just stops the split itself from adding to it, and the proper next step would be repeated stratified k-fold so the reported figure carries an interval rather than a single point estimate.

### *Q. Why CLAHE, and what does it actually do to the image?*

CLAHE is Contrast Limited Adaptive Histogram Equalization. It works on small tiles rather than the whole image, and within each tile it redistributes intensity so that local regions become distinguishable, with a clip limit that stops it from amplifying noise in near-uniform tiles. It matters for radiography because of how narrow the useful dynamic range is. A chest film is mostly dark lung with a small bright mediastinum, and global histogram equalisation would either wash out the lung fields or blow out the mediastinum. CLAHE instead reveals subtle local findings, faint consolidation, small nodules, and interstitial opacity, which is exactly the signal the classifier is looking for. The cost is real and I should mention it: contrast enhancement can amplify artefacts and noise along with genuine findings, so the same preprocessing that helps recall can also manufacture false texture, and that is a plausible contributor to some of the TB versus Pneumonia confusion.

### *Q. You describe normalisation two different ways. Which is it?*

Both, in sequence, and the write-up presents them as alternatives when they are actually two steps. First the image is converted to a tensor and scaled into the zero to one range, which is what the transform to tensor does. Then that tensor is standardised channel-wise with the ImageNet mean and standard deviation, the 0.485, 0.456, 0.406 and 0.229, 0.224, 0.225 vectors. The second step only makes sense because the model was initialised from ImageNet weights, because it is matching the input distribution those weights were trained under. If I were training from scratch the ImageNet statistics would be arbitrary and I would just use the zero to one scaling. So the zero to one step is the physical requirement and the ImageNet statistics are the transfer-learning requirement, and the documentation should say that rather than presenting the two as though they were competing descriptions of one step.

### *Q. You augment with random horizontal flips on chest X-rays. Is that not anatomically wrong?*

It is a fair objection and I would not defend it as clinically correct. Chest anatomy is strongly asymmetric, the heart sits on the left, the aortic arch curves to the left, and the gastric bubble is normally left-sided, so a mirrored radiograph is anatomically impossible. It also conflicts with my own clinical reading of the confusion matrix, because a model trained on mirrored films can partially resolve left-versus-right by memorising the flipped orientation of the cardiac silhouette, which is a shortcut that will not survive on a correctly-oriented film. I included it because it is standard in general computer-vision augmentation and it genuinely expands the training distribution, and I did not think it through in the medical-imaging context. If I were revisiting it I would drop the horizontal flip, or apply it only as a mild rotation, and I would re-run the comparison, because I cannot now claim how much of the reported accuracy it contributed. Worth noting the augmentation magnitudes also differ between my chapters, fifteen degrees and fifteen percent in one place, ten in another, and that is another inconsistency that should be reconciled.

### *Q. Why resize to 256 and then centre-crop to 224, instead of resizing straight to 224?*

Because a centre crop after a slightly larger resize preserves the effective resolution of the central anatomy instead of squashing the whole frame into the model's input. Radiographs are typically taller than they are wide, and the diagnostic content, the lung fields and the mediastinum, occupies the central portion, with dark margins and often collimation edges carrying no information. Resizing the long edge to 256 and then taking the central 224 discards the uninformative border while keeping roughly the native pixel density across the lungs. The cost is that it throws away the periphery, so a finding that happens to sit in the lateral costophrenic angle could be cropped out, and it makes the effective field of view dependent on the source image's aspect ratio, which differs between the archives I pooled. That is a preprocessing-consistency risk specific to a multi-source dataset.

### *Q. Why Adam at 1e-4 with ReduceLROnPlateau and early stopping, rather than discriminative fine-tuning learning rates?*

Adam with a single small learning rate and plateau decay was chosen for simplicity and because it converged without instability, at 1e-4 with decay by half on a three-epoch plateau and early stopping at five epochs monitoring validation loss, over a maximum of 50 epochs, batch size 32, with L2 weight decay at 1e-5. The standard alternative, which I did not use, is a discriminative schedule where early layers get a much smaller rate than the new classification head, which protects the generic ImageNet features from being destroyed by a small medical dataset. That is probably the more principled choice and it is what I would expect to help, so its absence is a simplification I made rather than a decision I can fully justify. What did help is the checkpoint rule, since selecting the weights at minimum validation loss and never touching the test set until the end is what keeps the headline number honest.

### *Q. You state CPU-only training in one place and list an RTX 3090 in another. Which is true?*

The RTX 3090 is true for training, and the CPU is true for the latency measurement, and I presented them in a way that reads as a contradiction. Training a DenseNet121 for 50 epochs on 21,000 images on CPU alone would have been impractical, so the backbone was trained on a 24 gigabyte GPU. The deployment target is different, and that is the whole point of the parameter choice: the trained weights are shipped as a 28.4 megabyte state dict and executed on an Intel Core i7 with no GPU at all, which is the configuration the 118 millisecond figure was measured in. So the correct statement is that training used GPU acceleration and inference is CPU-only, which is the normal arrangement for this kind of work and is exactly the property that makes the system deployable in a clinic without a GPU server. The sentence claiming a CPU-only environment for model development is simply wrong and needs correcting.

### *Q. Your write-up lists Grad-CAM as future work, but the system already produces heatmaps. Why?*

Because the system was implemented after that scope statement was written, and I did not go back to update the earlier chapters. The Grad-CAM overlays are in fact implemented, served as authenticated media and rendered beside the prediction in the results view, and the related chapter also lists explainability as a future extension, which is now equally out of date. This is the same class of problem as the dataset and augmentation inconsistencies, namely that the documented scope and the implemented system drifted apart as the work progressed and only part of the document was revised. It is worth flagging that this also affects the honesty of the limitations section, because the limitations no longer describe the final system.

**Closing note.** If a question is asked that you do not know, the correct response is to say so and then describe how you would find the answer, for example by reading the route decorator, checking the schema, or writing a test to observe the behaviour. An examiner is assessing whether you understand your own system; an honest boundary is far more convincing than a confident invention, and every answer in this bank is written on that assumption.